

EdgeBench: 揭开真实世界环境下学习的缩放规律

——中文翻译版——

EdgeBench: Unveiling Scaling Laws of Learning from Real-World Environments

原文作者: ByteDance Seed 翻译日期: 2026 年 9 月 29 日

翻译工程: 基于 prompt_translation.md 翻译规范

原文版本: 2026-07-07

摘要

摘要 (中文译版)

预训练阶段的缩放规律表明, 模型能力会随数据量与算力的增加而可预测地提升。然而, 智能体部署后从真实世界环境中学习的现象, 迄今仍远未被充分理解。本文分析了大语言模型智能体在 **134** 个真实世界任务上累计约 **38,000** 小时的交互数据, 首次以**对数-S 形 (Log-Sigmoid)** 这一简洁的函数形式刻画了环境学习的整体性能曲线, 并以极高精度拟合——决定系数 $R^2 = 0.998$ 。跨代际比较显示, 智能体的学习速度大约**每三个月翻一倍**。这一发现源自 **EdgeBench**: 一个包含 134 个超长视野真实世界任务的评测套件, 涵盖科学发现、软件工程、组合优化、专业知识工作、形式化数学与交互式游戏。每个任务在丰富、多层次的反馈下支持至少 12 小时的连续运行, 且均由领域专家精心构建。本文公开 51 个任务及完整评测框架, 以推动智能体如何从真实经验中学习这一研究方向。

原文摘要 (English Abstract)

Pretraining scaling laws reveal that model capability improves predictably with data and compute. But learning from real world environments after deployment remains far less understood. Analyzing roughly 38,000 hours of agent interaction with the environment across 134 real world tasks, we find, to the best of our knowledge, the first evidence that overall performance during environment learning follows a log-sigmoid scaling law with remarkably high precision, reaching $R^2 = 0.998$. Across model generations, we also find that agent learning speed roughly doubles every three months. This discovery stems from EdgeBench, a suite of 134 real world tasks with ultra-long horizons, spanning scientific discovery, software engineering, combinatorial optimization, professional knowledge work, formal mathematics, and interactive games. Each task sustains at least 12 hours of continuous agent operation under rich, multilevel feedback, and is built through substantial expert effort. We publicly release 51 tasks and our full evaluation framework to accelerate the study of how agents learn from real world experience.

项目主页: <https://edge-bench.org>

通讯作者: Shu Zhong zhongshu@bytedance.com

出版信息 原文于 2026 年 7 月 7 日提交至 arXiv(编号 2607.15651), 由 ByteDance Seed 团队发布。

1 引言

预训练阶段的缩放规律 [?, ?] 揭示了模型能力可随数据量与算力的增加而可预测地提升。这一洞见指导了过去数年大语言模型领域的诸多重要进展 [?, ?, ?, ?, ?, ?, ?, ?]。如今, 大语言模型正迈入“智能体 (Agent) 时代”, 它们被部署到日益多样的真实世界环境中, 能够在交互中尝试学习。然而, 这种“从环境中学习”是否同样服从简洁的缩放规律, 迄今仍属未知。

通过对 134 个真实世界任务上累计约 38,000 小时的智能体环境交互数据进行分析, 我们 (据我们所知) 首次发现了环境学习整体性能随交互时间遵循对数-S 形 (log-sigmoid) 缩放规律这一证据, 且拟合精度极高, $R^2 = 0.998$ 。

为什么研究智能体从环境中学习的能力? 真实世界的 AI 应用远不止模型在训练阶段学到的那些能力。一些必要知识从未出现在训练数据中——例如私有记录和内部工具。即便原始数据存在, 也往往缺失了背后的人类过程: 反复试错、对证据的解读、依据反馈做出的适应性调整——而这正是专家实际取得成果的方式。真实世界也从未静止: 人类知识持续前进, 新的工具、发现与问题不断涌现, 任何固定的训练语料都无法预见。因此, 智能体从环境中学习并提升任务表现的能力, 是 AI 系统大规模部署到真实世界的核心要素。

研究这种能力需要任务环境具备以下三个条件:(1) 接近真实使用场景;(2) 提供富有信息量的学习反馈;(3) 允许智能体通过交互进行足够长时间的学习。然而, 现有的评测基准要么缺乏这种反馈, 要么把智能体的运行时间限定为几分钟到几小时 [?, ?, ?, ?, ?, ?], 因此并不适合本研究的目标。为弥补这一空白, 我们构建了 **EdgeBench**: 一个由 134 个真实、多样化任务组成的评测套件, 横跨六个能力家族 (参见图 2)。每个任务运行在一个可执行的工作区中, 结合快速的本地探索与较慢的、针对提交产物的判官反馈, 以模拟真实工作流。智能体可在每个任务上持续工作至少 12 小时 (相比之下, Agents' Last Exam [?] 平均每个任务仅约 1 小时); 同时, 我们记录其提交并追踪表现随时间的演进。这些任务对人类专家而言也极具挑战性: 记录的专家投入平均为 57.2 小时/任务, 最高达 320 小时/任务。这使 EdgeBench 自然成为研究智能体如何在长视野下从环境中学习的理想测试平台。

借助 EdgeBench, 我们在约 38,000 小时的环境交互上评测了前沿智能体, 并归纳出四项核心观察:

1. 环境学习呈现出精确的对数-S 形缩放规律——在跨任务求平均的总体曲线、各任务家族、更长至 72 小时的交互视野下, 以及在使用早期轨迹预测后期性能时, 均呈现相同函数形式;
2. 对数-S 形规律的理论推导——我们提出一种理论, 将环境学习建模为在潜在任务图 (latent task graph) 上的“前沿扩张 (frontier expansion)”过程, 从而解释了为何基准求平均后的进度呈现观察到的对数-S 形;
3. 智能体学习速度大约每三个月翻倍——研究自 2025 年 9 月以来发布的前沿模型, 即可发现前沿智能体的环境学习速度正快速提升;
4. 学习动力学强烈影响长视野表现——长视野性能取决于智能体如何利用已积累的经验, 而不仅仅是尝试了多少次。连续使用经验优于独立重启, 更长上下文提升记忆能力, 而详尽的案例研究表明: 反馈能够将大量失败的试探转化为少数持久性的进展。

2 EdgeBench

2.1 设计目标

EdgeBench 的目标是衡量一个自主智能体能否在陌生的真实世界环境中**通过经验学习**。这对评测基准提出了两项现有评估无法满足的要求:

- **超长视野、多样化任务。**探索、策略修订、经验积累等学习行为需要足够的时间和复杂度才能显现。短任务通常依赖记忆而非学习来求解, 因此要衡量学习能力就必须使用长视野任务。此外, 学习是一种通用能力, 这些任务还需横跨多样化领域。
- **真实、多层次的反馈。**在实践中, 人类专家从丰富反馈中学习: 测试失败、实验结果、反常现象、权威判断等等。无法提供如此丰富反馈的基准, 既无法衡量学习, 也会让智能体无从知晓评测究竟奖励什么。我们需要近似真实世界的反馈, 才能衡量真正的、通用目的的学习能力。

第一项要求驱动了我们的任务分类体系 (详见 第 2.2 节): 跨六个能力家族共 134 个任务, 每个任务都被设计为至少 **12 小时**量级、可承受前沿模型运行的挑战。第二项要求驱动了我们的评测协议 (详见 第 2.3 节): 每个任务独立模拟其专属的“真实世界”切片, 提供**隔离的工作环境**与**判官环境**、本地智能体驱动的反馈、提交门控的判官反馈以及宿主端的轨迹测量。

2.2 任务分类体系

我们搜索满足两条标准的真实任务:(1) 性能上限足够高, 使任何现有智能体都无法将其饱和;(2) 工作流支持**连续学习**而非一次性完成。这一搜索与跨领域专家协作进行, 共识别出六个能力家族、产出 134 个精心挑选的任务 (图 2):

- **科学问题与机器学习 (Scientific Problems & ML,39 个任务)**。每个任务均使用来自真实科研工作者的研究数据与实验设置。领域专长不可或缺: 智能体必须提出假设、选择模型、在有噪声观测上验证、并迭代优化。许多问题是开放式的, 尚无已知最优解。
- **系统与软件工程 (Systems & Software Engineering,36 个任务)**。智能体在生产级代码库上工作, 单个任务可能需要数千行代码改动, 在最大的案例中涉及超过 10 万行。由于代码跨模块相互依赖, 智能体必须同时考虑跨模块耦合以及正确性与性能目标。
- **组合优化 (Combinatorial Optimization,19 个任务)**。这些开放式、以 NP-hard 问题为主的任任务中, 精确方法不可行, 进度依赖于设计、调参与迭代启发式搜索策略。即使强大的求解器也有在更多时间与反馈下提升的空间。
- **专业知识工作 (Professional Knowledge Work,19 个任务)**。这些任务复现金融、教育、医疗、法律等领域的真实白领交付物——通常需要三年以上经验的从业者约三个完整工作日才能完成。许多任务具备精心设计的评分细则和多轮交付反馈, 以近似真实客户审阅循环, 使智能体可从结构化批评中学习并迭代修订。
- **形式化数学与定理证明 (Formal Math & Theorem Proving,13 个任务)**。这些任务位于数学难度的前沿, 需要在 Lean 中构建大规模机器可检查证明, 将深邃的数学洞察与大量形式化验证工程相结合。大部分任务为 EdgeBench 全新设计, 以支持迭代进展: 智能体获得结构化的中间指引, 并可增量扩展局部证明。
- **交互式游戏与模拟器 (Interactive Games & Simulators,8 个任务)**。这些是为人类玩家设计的真实游戏, 熟练的人类通常需要数十小时才能掌握机制。状态空间极大且每局游戏程序化地不同, 因此智能体面临强大的分布外 (**Out-of-Distribution, OOD**) 压力, 必须通过跨多个 episode 的高频交互开发并优化策略。

任务排除说明: 主要难度来自视觉理解 (尤其是 GUI 操作) 的任务被排除。当成功取决于视觉骨干而非迭代推理时, 学习能力与感知能力难以分离。

2.3 反馈循环与评测协议

真实世界的工程与研究 workflow 很少提供单一的“最终答案”检验。实际上, 从业者通过**两条互补的反馈循环**迭代: 一条用于探索、调试与精炼的**快速本地循环**, 以及一条通过部署、同行评审、基准评测或干系人反馈进行权威校准的**较慢外部循环**。本地循环促成快速进展, 外部循环防止对可见检验的过拟合, 并揭示开发者自身测试未覆盖的失败。

EdgeBench 采用这种“双循环”结构来衡量学习而非端点成功 (参见图 3)。内循环是本地且由智能体驱动: 智能体可检视可写工作区、运行测试或模拟器、观察错误、修订产物。外循环由判官中介: 提交的产物将基于隐藏用例或私有评分标准进行评估, 返回校准后的分数、判定或诊断。跨任务家族, 这一模式通过不同机制实例化: 软件任务使用测试与性能分析器, 科学任务使用开发拆分与验证器, 优化任务使用本地测试器与隐藏随机种子, 定理证明使用证明检查器的状态, 游戏使用 episode 分数, 专业知识工作使用评分细则。

该协议由一个**隔离的工作—判官评测沙箱**实现。在运行期间, 智能体在一个容纳任务材料与本地验证工具 (但不包含隐藏评测资产) 的**工作容器 (Work Container)** 内工作。智能体可主动将其当前产物提交到一个独立的**判官容器 (Judge Container)**, 该容器运行隐藏评测并返回任务定义的反馈。宿主端的判官服务器负责中介这一外循环, 包括提交队列、冷却时间、身份认证以及支持长时评测的异步评分, 使智能体在提交作业评分的同时继续工作。附录 C 给出了一些促使我们采用隔离与提交中介设计的失败模式示例。

为进行轨迹测量, 评测沙箱还会按固定间隔执行宿主端自动评测。这些快照通过隐藏判官评分并被记录用于分析, 但**结果不会展示给智能体**。这使得 EdgeBench 即便在两次显式提交之间也能衡量改进, 同时保留了“智能体可见反馈”与“评测者专属测量”之间的区分。

3 真实世界环境学习的缩放规律

经典预训练缩放规律 [?, ?] 将语言模型损失建模为训练规模的幂律函数, 包括预训练数据量 [?, ?]。相比之下, 基准性能则是模型能力的“任务级”读数, 受任务、样本与评分标准引入的难度阈值所塑造。先前的工作 [?, ?, ?, ?] 发现, 在预训练规模扩展下, 基准性能可用对数-S 形曲线良好描述。

除了从人类收集的训练数据中学习, 现代智能体 (如 GPT-5.5 [?], Claude Opus 4.8 [?]) 还可在部署后继续从环境中学习。在单个任务上, 它们可以通过交互获取新信息并随时间提升性能。然而, 这种学习形式是否同样服从简洁的缩放规律, 仍不清晰。EdgeBench 提供了 134 个多样化的真实世界任务, 具备可执行环境、富有信息量的反馈以及至少 12 小时的交互视野, 使我们得以研究智能体如何通过与环境交互提升性能。通过在 134 个多样化真实任务上分析五个前沿智能体约 **38,000** 小时的环境交互, 我们发现**对数-S 形曲线能够精确拟合环境学习性能**。因此, 环境学习与预训练数据学习诱导出了相同的数学缩放形式。

3.1 从任务轨迹到可预测的缩放曲线

实验设置。 我们用五个前沿模型评估 134 个 EdgeBench 任务: Claude Opus 4.8 [?], GPT-5.5 [?], GPT-5.4 [?], GLM-5.1 [?, ?] 以及 DeepSeek-V4-Pro (preview) [?]。每个“任务-模型”配对运行三次独立的 12 小时试验, 并记录完整提交轨迹。GPT 系列模型使用 Codex 与 256k 紧凑上下文窗口运行; GLM-5.1 与 DeepSeek-V4-Pro 使用 Claude Code 与 200k 紧凑窗口运行。Claude Opus 4.8 主

要使用 1M Claude Code 紧凑窗口运行; 此外我们在 第 5.3 节中加入了一个 200k vs. 1M Opus 消融实验。

逐任务轨迹的多样性。 图 4 展示了 18 个代表性任务的逐任务学习曲线, 展示了不同模型如何在同一任务上随时间改进。所有任务的完整曲线集合见附录 G.5。任务横跨多样化领域, 表现出异构的学习动力学: 轨迹范围从平滑渐进增益到长平台、突变突破乃至不规则回退。

聚合曲线揭示共同结构。 虽然这些个体曲线在任务与模型间高度异质, 但其跨任务平均曲线却出乎意料地平滑且具有共同结构。受先前在预训练扩展下对对数-S 形基准拟合的启发 [?, ?, ?], 我们用以下三参数对数-S 形模型拟合平均后的环境学习曲线:

$$S(t) = \frac{S_{\max}}{1 + (t_{\text{mid}}/t)^{\beta}}, \quad (1)$$

其中 t 为已用交互时间, $S(t)$ 为迄今最佳性能 (best-so-far performance)。拟合参数作为聚合学习曲线的经验性描述符:

- $S_{\max}$ ——可达到的分数上限;
- t_{mid} ——曲线达到该上限一半时所对应的交互时间;
- β ——控制在对数时间下进度集中的陡峭程度。

因此, 较小的 t_{mid} 意味着模型更快达到可达到分数的主体部分; 较大的 β 对应更陡的学习跃迁。第 3.2 节显示这种简洁函数形式能极其精确地拟合经验学习曲线。

3.2 对数-S 形曲线以惊人精度拟合环境学习

我们发现对数-S 形拟合在所有测试设置下都精确且稳健:

1. **对数-S 形曲线精确拟合 134 任务的平均值, 适用于所有五个模型。** 如图 1 所示, 在对 134 个任务求平均后, 拟合的对数-S 形曲线紧密跟踪每个模型的 12 小时学习轨迹。拟合在所有五个模型上都相当紧密, $R^2 \geq 0.997$ 。
2. **相同的对数-S 形形式在异构的任务家族间持续成立。** 图 5 重复地对 EdgeBench 中六个能力家族分别进行拟合。这些家族需要不同的知识、锻炼不同的能力, 且呈现明显不同的聚合学习曲线。然而, 每个家族仍可被相同的对数-S 形形式良好描述——即使在任务数较少、均值轨迹噪声较大的小型家族中也成立。
3. **在显著更长的交互视野下拟合仍然精确。** 图 ?? 将分析扩展到主 12 小时窗口之外, 达到 28 小时与 72 小时的交互视野。受资源所限, 28 小时拟合覆盖 80 个任务与四个模型, 72 小时拟合覆盖 18 个任务与两个模型。相同的对数-S 形结构在两种更长设置下保持稳定, 每条拟合曲线均达到 $R^2 \geq 0.993$ 。
4. **对数-S 形规律展现预测能力。** 图 6 通过仅用前 6.5 小时拟合每条 12 小时聚合曲线来验证这一预测能力, 然后在 6.5–12 小时的留出观测上评估外推。所有五个模型下, 外推曲线仍贴近后期的观测轨迹, $R^2 \geq 0.997$ 、RMSE 小于 1.0 性能点。

这些精确拟合引出两个问题: 对数-S 形真的就是“对的”曲线吗? 还是任意 S 形都行? 如此简洁的规律究竟从何而来?

对数-S 形在常见 S 形曲线中拟合最佳。 许多饱和增长过程都是 S 形的 [?]. 因此我们将对数-S 形与其他常见 S 形比较: 对数-Probit [?], 对数-Gompertz [?] 和基于原始时间的 Weibull 累积分布函

数 (CDF) [?] 以及一个两参数对数-线性基线; 对基于对数时间的曲线族, 自变量设为 $x = \ln t$ 。我们对 12h、28h、72h 全窗口拟合每一族并汇总误差。如表 1 所示, 对数-S 形族取得最低 RMSE, 而对数-线性基线显著较差。因此经验信号稳健地呈 S 形而非依赖于单一连接函数, 也不能仅由对 $\ln t$ 的线性改进解释。附录 E 从机制层面进一步论证我们为何仍偏好对数-S 形。

曲线族	函数形式	RMSE
对数-S 形	$S(t) = \frac{S_{\max}}{1 + (t_{\text{mid}}/t)^\beta}$	0.390
对数-Probit	$S(t) = S_{\max} \Phi\left(\frac{\ln t - \mu}{\sigma}\right)$	0.398
对数-Gompertz	$S(t) = S_{\max} \exp[-\exp\{-c(\ln t - x_0)\}]$	0.402
Weibull CDF	$S(t) = S_{\max}(1 - \exp\{-(t/\lambda)^\beta\})$	0.404
对数-线性	$S(t) = a + b \ln t$	0.717

表 1. 三参数 S 形族与两参数对数-线性基线在全窗口上的拟合误差。RMSE 以 0–100 分制下的“性能点”度量, 越低越好。

该规律源于任务总体。单个任务的轨迹噪声大且独特, 然而图 7 显示随着求平均的任务数增加, 对数-S 形拟合变得越来越精确: 残差误差从 1 个任务单调下降到所有 134 个任务。因此, 这一简洁的缩放规律是一种涌现的总体级规律, 只有跨众多多样化任务时才显现锐利形式, 而在任何单一任务内部则不成立。

3.3 对数-S 形规律的理论

经验拟合展示出稳健的对数-S 形形状, 但仅凭拟合本身并不能解释这一形状为何应该出现。我们提出一种理论模型: 环境学习是潜在任务图上的前沿扩张过程。在这一视角下, 每个任务被建模为一个“得分单元”的潜在图, 已解锁的单元对锁定的邻居施加影响以解锁它们, 当已解锁与已锁定得分节点之间的“前沿”推进时, 进度即发生。附录 D 给出了完整推导, 这里我们总结其机制。在本节中, 我们使用 $u = \log t - \log t_{\text{mid}}$ 作为时间轴的坐标变换。

环境学习是前沿扩张过程。对单一任务, 设任务分数由许多得分单元构成, 代表任务图 G 上的节点 i , 权重为 w_i , 归一化得分权重为 $\mu_i = w_i / \sum_i w_i$ 。令 $n_i(u) \in \{0, 1\}$ 指示单元 i 在时间 u 是否已解锁, 则获得的归一化分数为:

$$x(u) = \sum_i \mu_i n_i(u).$$

在任务图 G 上, 边权重 $K_{ij} \geq 0$ 度量已解锁源单元 j 对解锁目标单元 i 的帮助程度。因此, 锁定的单元 i 接收影响场:

$$h_i(u) = \sum_j K_{ij} n_j(u).$$

若锁定单元以与该场成比例的期望速率随机解锁, 则在当前状态条件下:

$$\frac{d}{du} \mathbb{E}[x(u) | n(u)] = \eta \sum_{i \in L(u)} \sum_{j \in U(u)} \mu_i K_{ij}. \quad (2)$$

因此, 期望分数增长率恰好等于从未解锁单元集 $U(u)$ 到锁定单元集 $L(u)$ 的加权前沿切割。

前沿过程以 $x(1-x)$ 的速度推进。精确的前沿切割仍依赖于任务图结构。均值场近似假设在聚合层面, 每个宏观的“未解锁--锁定”切割都近似具有“乘积测度”的影响:

$$\sum_{i \in L} \sum_{j \in U} \mu_i K_{ij} \approx \kappa \mu(L) \mu(U),$$

其中 $\mu(A) = \sum_{i \in A} \mu_i$ 。结合式 (2), 得到:

$$\frac{dx}{du} = \beta x(1-x), \quad \beta = \eta \kappa. \quad (3)$$

两个因子都有直接的物理解释: 已解锁的得分质量提供可复用能力, 而锁定的得分质量衡量剩余的改进机会。附录 D 中的 D.2 节使用“加权切割混合”条件形式化了这一近似, 该条件弱于“假设所有边权重各自相等”。

有效时间坐标是对数的。前沿方程写在有效的任务图坐标 u 上。 u 近似 $\log t$ 的自然原因是自相似图结构¹。如果任务难度的每一加法增加暴露乘法更大的相关图结构, 那么遍历图所需的搜索量随难度尺度指数增长。若跨时间尺度的搜索努力近似为常数, 则时间 t 所能达到的难度尺度满足:

$$u \sim \log t.$$

将此坐标代入前沿方程式 (3) 即得:

$$\frac{dx}{d \log t} = \beta x(1-x). \quad (4)$$

求解前沿方程。分离变量于式 (4) 得:

$$\log \frac{x(t)}{1-x(t)} = \beta \log \frac{t}{t_{\text{mid}}},$$

其中 t_{mid} 选取使得 $x(t_{\text{mid}}) = 1/2$ 。因此:

$$\frac{1}{S_{\text{max}}} = \frac{1}{1 + (t_{\text{mid}}/t)^\beta} \implies S(t) = \frac{S_{\text{max}}}{1 + (t_{\text{mid}}/t)^\beta}. \quad (5)$$

即重新得到式 1。

基准均值比单个任务更平滑。上面的论证刻画的是极限前沿漂移。它并不意味着每个有限任务都会以肉眼可见的平滑 S 形推进。仅有少量得分单元的任务可能呈现长平台与突然跳跃。经验缩放规律是关于任务聚合曲线的命题: 若许多独立评测的任务各自遵循近似的前沿动力学, 则求平均会消除有限任务的锯齿状。当残余任务中点与任务速度足够集中时, 聚合曲线将退化为单一对数-S 形:

$$\frac{1}{M} \sum_{b=1}^M x_b(u) \approx \frac{1}{M} \sum_{b=1}^M \frac{1}{1 + e^{-\beta(u-\delta_b)}} \xrightarrow{P} \frac{1}{1 + e^{-\beta u}}.$$

附录 D 中的 D.3 节在“逐块切割混合”、“跳跃噪声平均消失”、“中点对齐”与“速度集中”等假设下给出了精确表述。

¹这让人联想到物理复杂系统中的自组织临界与临界现象——其行为不由单一特征尺度支配 [?, ?]。

拟合参数的解读。 拟合速率 β 可自然地解读为对数时间下的**有效前沿传播速度**。较大的 β 意味着分数解锁发生在更窄的交互尺度区间内, 产生由低到高的更陡跃迁; 较小的 β 意味着进度分散于更多的乘法时间上, 产生更平缓的学习曲线。拟合上限 $S_{\max}$ 应解读为拟合区间内**可达的分数支撑**, 而非性能绝对意义上的上界。

适用范围与局限。 对数-S 形规律并非对每个环境学习过程都成立。当任务图含有强瓶颈、分散的任务中点、异构的前沿速度或非自相似图结构时, 该规律可能失效。这些失败模式澄清了理论的范围: 对数-S 形在“进度表现为在近似分形结构图上充分混合的前沿扩张过程”时最自然。在此意义上, 从环境中学习衡量的是智能体能否**将多样化反馈转化为可复用的结构**, 从而加速后续发现。我们认为这是“环境学习值得度量与扩展”的核心原因。

4 智能体学习速度大约每三个月翻倍

前沿智能体现在能够通过与任务环境的交互来提升表现。这引出一个独立问题: 较新的模型是否学习得更快? 我们通过比较各智能体在**固定交互预算**下获得的性能增益, 按模型发布日期衡量。

4.1 实验设计

分离先验知识与环境学习。 高分可能反映模型已经掌握的知识, 而非在运行中学到的内容。为减少这一混淆, 我们从 EdgeBench 中选取一个 18 任务的子集, 这些任务上模型表现出相近的首次尝试性能。由于起点相当, 后续增益提供了对环境学习的更纯净度量。我们将“任务学习速度”定义为在**固定两小时预算下的平均性能增益**。

评测协议。 我们评测自 2025 年 9 月 (前沿系统开始具备持续自主运行能力时) 至当前评测窗口期间发布的前沿开源与闭源 AI 系统。每个模型每个任务运行三次。GPT 系列模型使用 Codex 运行; 所有其他模型使用 Claude Code 运行。如图 9 左图所示, 模型在该 18 任务子集上从可比的首次尝试性能起步, 平均 6.87 ± 0.97 。

4.2 跨模型代际的学习速度

图 8 报告了在固定 18 任务子集上的两小时评测结果。纵轴以对数尺度衡量智能体学习速度, 定义为两小时内的性能增益。为估计前沿趋势, 我们以深色标记突出每个发布日的**前两名模型**, 然后对这些前沿点拟合线性趋势, 95% 置信区间显示为阴影带。

图 8 显示了近期模型代际间学习速度的快速提升。从 2025 年 9 月的 GPT-5-Codex 到 2026 年 4 月的 GPT-5.5, 学习速度在 221 天内增长约 **8 倍**。对前沿模型的对数-线性拟合很好地捕捉了这一趋势, 对应于**大约每三个月翻一倍**。图 9 显示该改进并不能简单地用“更频繁提交”解释。提交频率 (右图) 变化不均匀: 较新的 GPT 模型提交更频繁, 而其他家族则不然。中图讲述了一个不同的故事: 较新的模型能将**更大比例**的提交转化为“迄今最佳”改进。因此, 该趋势反映的是每次交互中**更有效的学习**, 而不仅仅是更多尝试。

5 环境学习动力学分析

我们研究前沿模型如何在长视野下利用环境反馈进行表现与学习。我们评测五个前沿模型: Claude Opus 4.8、GPT-5.5、GPT-5.4、GLM-5.1 与 DeepSeek-V4-Pro(DS-V4-Pro)。本节包含四项分析。第 5.1 节从聚合、家族、任务与提交层面比较前沿模型; 第 5.2 节测试累积经验是否相比独立重启具有额外价值; 第 5.3 节检验更长上下文是否在长视野交互中提升性能; 第 5.4 节跟踪一个科学任务以

模型	总分 ↑						12 小时分家族分数 ↑					
	@2h	@4h	@6h	@8h	@10h	@12h	科学	代码	优化	知识	数学	游戏
Opus 4.8	39.0	45.7	48.1	49.8	50.9	51.3	48.5	67.4	36.5	47.0	55.0	39.3
GPT-5.5	36.8	42.1	44.5	46.3	47.6	<u>48.4</u>	44.3	65.0	33.6	45.7	50.0	39.1
GPT-5.4	29.7	34.0	36.5	38.0	38.9	39.3	<u>33.5</u>	54.1	27.9	38.8	40.8	29.0
GLM-5.1	26.0	30.4	32.9	34.9	36.5	37.4	33.8	50.9	<u>26.4</u>	<u>43.5</u>	24.6	29.3
DS-V4-Pro	23.3	27.1	29.0	29.9	30.9	31.0	30.0	43.0	21.5	37.0	14.1	16.9

表 2. 聚合排行榜。总分报告于每个时间预算; 分家族分数报告于 12 小时预算。**加粗**为该列最佳值,下划线为次佳。

展示智能体改进的具体展开方式。附录 A 报告了 “/goal 模式” 与 “Ralph 循环” 在沙箱级别的额外续接消融。

5.1 前沿模型比较

设置。 我们沿用 第 3.1 节的设置, 并从两个角度分析 12 小时轨迹:(1) 聚合、家族与任务层面的表现; 以及 (2) 提交效率, 即智能体的提交有多频繁地改进当前最佳。

表现比较。 表 2 给出聚合表现。Claude Opus 4.8 在 12 小时达到 **51.3** 分, 其次是 GPT-5.5 的 **48.4**。GPT-5.4 与 GLM-5.1 组成下一梯队 (39.3 与 37.4),DS-V4-Pro 取得 31.0。家族级结果与聚合排名大体一致:Claude Opus 4.8 在每个家族均值上领先,GPT-5.5 在游戏上尤其接近且总体排名第二。详细任务表现见附录 G.6,* 标记少于三次有效运行的单元格。

提交效率。 我们对每个智能体的每次提交计数, 当其将 “迄今最佳” 分数推至新高时标记为 “有效”。图 10 比较了各模型 12 小时的有效提交率与最终表现。做出更多有效提交的模型通常表现更好, 但 “更多提交” 并不自动带来更好的最终结果。**Claude Opus 4.8** 取得了最佳最终表现, 即便其提交频率低于 GPT-5.5;GPT-5.4 拥有最高有效提交率, 但仍落后于前两名。因此, 进度不仅取决于智能体改进分数的频率, 还取决于改进是否大、可靠且可复用。较强的智能体会更审慎地使用反馈: 构建可提交的基线、保留当前最佳方案、做出有针对性的改动, 并利用反馈保留收益或回滚失败。较弱的智能体则更倾向于过度信任本地代理、捆绑无关修改或在反馈已排除某方向后仍继续广泛探索, 从而降低采样效率。

5.2 智能体确实从经验中学习, 而非仅仅重复采样

动机。 “迄今最佳” 曲线不断上升本身并不能证明智能体在 “学习”。运行更长时间同样增加偶然撞到好解的机会: 在足够多的独立尝试下, 最佳的尝试仅凭运气就能提升。因此, 我们测试智能体的累积经验是否在相同总时间预算下, 比单纯重复采样带来额外价值。

设置。 我们在 17 个任务上为 Opus 4.8 给出相同的 12 小时预算, 并比较两种花费方式: 有经验 vs. 无经验。

- **有经验:** 智能体连续运行一次, 保留其工作区、产物与反馈历史, 使经验跨整个运行积累。
- **无经验:** 同样预算被切分为 $n = 6$ 次独立的 $\tau = 2$ 小时尝试, 各次之间丢弃所有状态, 只保留最佳结果, 因此每次尝试从头开始, 任何增益只能来自重复采样。

在经过时间 $t = k\tau$ 时,二者对比即“一次连续运行 t 小时后”与“使用相同总时间的 k 次独立尝试中最佳者”。附录 G.1 详细描述了如何估计各曲线。

结果。 图 11 显示经验带来明显收益: 在相同时间预算下,“有经验”曲线始终高于“无经验”基线。在 12 小时,其达到 43.0 vs. 36.1, 提升 +6.9。因此,该改进不能单独由重复采样解释: 累积并复用任务经验驱动了独立重启所无法达到的进度。

5.3 更长上下文对性能有多少提升?

动机。 第 5.2 节表明累积经验能提升性能。一个悬而未决的问题是,在长运行中应当如何保留经验。使用长上下文是一种自然的方法。然而,前沿智能体框架也可通过工作区文件、压缩、进度备忘以及类记忆制品,在模型上下文之外维护状态。因此,在这些外部状态通道已可用的情况下,扩展上下文窗口是否仍能带来额外收益(若有,多大)并不明确。

设置。 我们在 42 任务子集上,在相同的长视野评测协议下比较 200k 上下文的 Opus 4.8 与 1M 上下文的 Opus 4.8。

结果。 更长的上下文在整个 12 小时窗口上带来稳定的多点提升(图 11 右)。1M 上下文的 Opus 4.8 在每个检查点上都高于 200k 版本,两条轨迹近乎平行: 差距在 2h 时为 +5.8, 到 12h 仅略微收窄到 +4.4, 且两条曲线均很好地符合相同的对数-S 形形式。因此,即便具有相同的工作区与沙箱状态,更长的上下文窗口在整段视野上都提供了稳定优势,且仅有略微收窄的趋势。

5.4 案例研究

设置。 我们考察引力波重构任务。基于首篇 GW150914 检测论文 [?], 智能体需要从 LIGO 应变数据复现已发表的信号分析。目标分三组输出: H1/L1 波形、H1/L1 时频谱、以及源动力学(速度/分离度)曲线。判官权重将五个组件分数设为: 每个波形 0.15, 每个时频谱 0.20, 以及速度/分离 0.30。我们以 30 分钟评测间隔与 120 秒提交冷却运行 Codex 智能体,并禁用网络访问。智能体进行了 224 次显式提交,沙箱增加了 23 次自动评测,运行在 12 小时预算时超时。

轨迹揭示了稀疏但结构化的“诊断--编辑--评测”循环。该循环之所以稀疏,是因为大部分提交是探索性试探: 只有 224 次智能体提交中的 27 次将“迄今最佳”分数提升至少 0.1 个百分点。其结构性在于反馈反复改变了智能体的搜索方向: 智能体首先使任务可度量,然后将未解决的错误拆分为更小的搜索,识别主要瓶颈并持续围绕其搜索,最后保留一个可行方案同时修复剩余错误。这些模式展示了“许多失败的试探仍能产出少量累积改进”。

详细阶段:

- 智能体首先使问题可度量,然后再使其更好。第一次有效提交将一项定义不清的分析任务转换为可打分的流水线,但反馈暴露了薄弱的源动力学。随后智能体在最初 11 次提交中稳定该流水线并替换一个噪声较大的频率估计,产生三次有意义的更新与 +4.5 个百分点的提升。
- 直接修复停滞时,智能体将失败分解为可搜索的子问题。智能体并未将波形不匹配视为一个不透明错误,而是将参考锚定、时频定位与探测器对齐分离开来。在 40 次信号搜索提交中,这种分解产生七次有意义的更新,将最佳分数提升至 52.3。

- 识别主要瓶颈使智能体持续高效搜索。在广义信号处理编辑进入平台期后, 组件反馈将“速度/分离”识别为最显著差距。智能体在源质量校准内继续搜索, 而非重写整条流水线; 在 4–5 小时窗口中, 17 次提交与 5 次有效更新将源动力学从 64.2 提升至 89.0, 造成整个运行中最大的跳跃。
- 找到稳定方案后, 智能体保留核心并仅修复剩余错误。在最后几小时, 大多数残差编辑无法迁移。智能体保留源模型固定, 转而测试有针对性的残差校正、相位对齐与窄带校正。这些有效更新将 H1 波形组件分数从约 47 提升至 95, 聚合最佳分达到 67.0。

通过不均匀跳跃而非平滑爬升实现改进。在 247 次打分评测中, 最佳分数在 0–100 分制上从 42.8 升至 67.0。图 12 展示了 7 个代表性里程碑。这些跳跃对应上述行为模式: 智能体首先使任务可打分, 接着定位信号、改进源动力学, 最后修复剩余的 H1 波形错误。详细里程碑阶段与最终子分构成见附录 G.2。

6 相关工作

基准	# 任务	视野	场景	自演化
MMLU [?]	15,908	短	知识问答	否
AIME [?]	30	短	高中数学竞赛	否
GDPval Gold [?]	220	短	专业交付物	否
SWE-bench Verified [?, ?]	500	短	Issue 修复	否
Terminal-Bench 2.0 [?]	89	短	终端工作流	否
Continual Learning Bench [?]	6	短	受控任务序列	是
CL-bench [?]	1,899	短	上下文学习	否
FrontierCode [?]	150	N/A	生产代码改动	否
NL2Repo-Bench [?]	104	中	仓库生成	否
Agents' Last Exam [?]	152 (公开)	中	计算机使用工作流	否
MLE-bench [?]	75	长	ML 工程	是
MLS-Bench [?]	140	长	ML 研究	子集
Frontier-Eng [?]	47	长	工程设计	否
Frontier-CS [?]	156	长	开放式 CS 任务	否
AutoLab [?]	36	长	研究/工程优化	否
FrontierSWE [?]	17	长	软件工程/性能调优	否
EdgeBench(本文)	134	超长	软件工程, 科学/ML, 专业工作, 优化, 形式化证明, 游戏	是

表 3. 与代表性基准的比较。“视野”刻画的是单实例任务合约, 非套件总运行时间。仅当性能被显式地相对于资源轴 (如时间或样本数) 绘制时, 才计为“自演化”。

现有基准覆盖了智能体能力的大部分, 但很少衡量智能体如何在一次运行内改进。闭卷问答、数学与编程基准 (如 MMLU [?], AIME [?], HumanEval [?]) 以及端点补丁基准 (如 SWE-bench [?]) 评估的是最终答案或最终补丁。更多的智能体与面向工作的基准, 包括 GDPval [?], Agents' Last Exam [?] 和 FrontierCode [?], 将任务接口扩展到专业交付物、计算机使用工作流或生产代码改动。然而, 其报告的中心指标仍是端点度量: 任务成功率、产物质量、通过率或生产就绪度。EdgeBench 转而衡量随时间改进的轨迹。

某些基准研究学习,但聚焦于受限领域与较短视野。CL-bench [?],EvaLearn [?] 和 Continual Learning Bench [?] 评估模型能否从静态上下文或静态信息流中改进——其内容主要并非由智能体在单任务内的行为塑造。迭代优化基准,如 MLE-bench [?],MLS-Bench [?],Frontier-Eng [?],Frontier-CS [?] 与 ALE-Bench [?],进一步纳入了重复尝试、经验反馈或可见的指标。最接近的长视野智能体对比基准是 AutoLab [?] 与 FrontierSWE [?]:两者均评估能反复编辑可执行产物并吸收反馈的智能体。EdgeBench 的差异在于覆盖了更广泛的可执行领域、采用日级任务合约,并在所有任务上一致地应用相同的轨迹度量。

缩放规律在先前工作中已被广泛研究,但从多样化真实环境中学习仍未被充分探索。经典缩放规律研究预训练,将损失或基准性能与模型规模、数据与算力联系起来 [?, ?];后续工作研究了模型规模扩展下有界基准性能曲线 [?, ?, ?, ?]。测试时缩放规律也已在多种推理时方法下被研究:纯重复采样 [?, ?, ?]、搜索、修订或验证器引导的计算分配 [?, ?],以及推理模型中的长链思维推理 [?, ?]。智能体测试时缩放已在计算机使用与浏览智能体中被观察到 [?, ?],通过交互长度缩放开展研究 [?],并经由长视野人-智能体比较被重新审视 [?]。这些研究更接近我们的设置,但覆盖的领域更窄、任务更少,且未建立跨域的缩放规律。强化学习缩放也与之相关,因为它研究从环境反馈中学习 [?, ?];而 EdgeBench 测量的是已部署智能体轨迹中的已用交互时间。从任务反馈中做上下文学习 (In-Context Learning, ICL) 相对廉价,可跨许多可执行环境反复评测;而大规模强化学习运行代价高昂,通常覆盖环境较少。EdgeBench 更广的环境覆盖或许正是聚合曲线稳定到足以揭示缩放规律的原因。

更详细的逐基准讨论见附录 F。

7 结论

本文介绍了 EdgeBench——一个用于研究智能体如何在日级视野下从真实世界环境中学习的基础。在 134 个多样化可执行任务与约 38,000 小时的环境交互上,我们发现聚合学习轨迹遵循与交互时间的精确对数-S 形关系。同一形式跨任务家族保持稳定,在更长视野下保持稳定,并支持从早期轨迹预测后期性能。我们还发现智能体的学习速度在近期前沿模型代际间快速提升。

这些结果表明,从环境中学习并非一系列零散的任务结果,而是可度量的缩放对象。与许多聚合能力曲线不同,环境学习轨迹揭示了产生进步的中间尝试、反馈与修订。这使得 EdgeBench 不仅可用于为智能体排名,还可用于研究智能体如何获得并复用经验。

更广泛地说,我们观察到的规律性表明:部署后从丰富环境中学习或许值得享有与预训练相同的系统性缩放关注。

附录目录

- 附录 A 评测沙箱
- 附录 B 服务与 API 稳定性
- 附录 C 评测作弊
- 附录 D 对数-S 形规律的完整推导
 - D.1 预备:潜在能力图与可达支撑
 - D.2 环境学习作为前沿扩张过程
 - D.3 多任务聚合揭示平滑的对数-S 形规律
 - D.4 图自相似性诱导对数时间尺度
 - D.5 讨论与局限

- 附录 E 缩放规律形状的更多讨论
- 附录 F 附加相关工作
 - F.1 不适用于衡量自演化的基准
 - F.2 适用于衡量学习或自演化的基准
 - F.3 LLM 与智能体的缩放规律
- 附录 G 附加基准与实验细节
 - G.1 “有/无经验”曲线的估计
 - G.2 引力波案例研究细节
 - G.3 沙箱级续接消融
 - G.4 逐任务设计备注
 - G.5 逐任务学习曲线
 - G.6 逐任务分数表
- 附录 H 致谢

A 评测沙箱

标准的单元测试沙箱不足以应对日级运行。该基准必须隐藏评测资产、支持多小时的重复提交、即便在智能体未显式提交时也能衡量进度，并在本地机器与集群上均能稳定运行。我们围绕三种机制构建：**隔离的工作/判官环境**、**以在线判官为模型的反馈循环**、以及**宿主端的进度跟踪**。

双容器架构。 每个任务被具体化为两个任务专用的 Docker 镜像：

- **工作镜像 (Work Image)** 包含骨架代码、文档与本地验证工具，但不包含隐藏评测资产。智能体仅在此环境中操作。
- **判官镜像 (Judge Image)** 包含隐藏评测资产、评分脚本与评测命令。智能体永远不会暴露于该环境。

评测时，智能体的代码被复制到一个临时判官容器中，该容器运行隐藏测试，仅返回任务定义的结构化反馈，然后被销毁。这种分离防止了智能体检视或修改隐藏评分器，同时仍允许现实的迭代开发。

判官服务器与外循环。 外循环由一个宿主端 HTTP 判官服务器中介。智能体将其当前方案提交到服务器，后者将提交入队、运行判官容器、解析结果，并返回诸如通过率、分数、逐测试判定或诊断之类的反馈。对于长时评测，服务器支持**异步评分**，使智能体在提交作业被评分的同时继续工作。服务器还强制实施**提交预算**、**冷却时间与会话认证**。

长视野执行与测量。 SForge 沙箱结合了宿主端自动评测与**停止钩子 (stop-hook)** 与**自动恢复 (auto-resume)** 机制以支持日级运行。自动评测定期快照并通过特权通道对智能体工作区进行评测；这些结果被记录用于轨迹分析，但不展示给智能体。停止钩子与自动恢复机制减少了由自愿退出、崩溃、上下文限制或瞬时 API 故障造成的过早截断。

运维支持与安全保障。 SForge 还提供了用于运行与审计大规模实验的实用基础设施。一个 Web 仪表板记录分数轨迹、提交历史、差异与对话追踪，用于实时监控与事后比较。同一任务规范可在本地 Docker 后端或 Kubernetes 后端上运行，以支持集群级评测。为保持基准完整性，SForge 结合了

工作/判官隔离、提交过滤、网络隔离与会话级身份认证,使智能体能接收反馈而无法访问隐藏测试、特权历史或自动评测结果。附录 C 描述了我们在基准开发过程中观察到的任务设计失败模式以及相应的缓解措施。

B 服务与 API 稳定性

将单个智能体维持 12 小时以上,比短期评测更严重地考验 API 服务:日级运行必须保持模型、其上下文与工具调用不中断,该窗口内的任何服务事件都可能截断或退化轨迹。因此,我们的长视野任务不可避免地将**服务稳定性**折入所测量的内容。我们认为这是合适的,而非需要工程手段消除的混淆项:被部署以在真实世界工作数小时的智能体,正面临完全相同的服务现实,因此用于长视野学习的基准也应当反映这些。

图 13 对两个 GPT 模型进行了量化:将服务端事件率(按活跃运行时间归一化)与已用时间作图。GPT-5.4 比 GPT-5.5 经历了显著更多的基础设施与 API 中断,尤其在六小时标记之后。这对 GPT-5.4 带来两点提示。首先,几个少于三次有效运行的逐任务单元格(分数表中以 * 标记)涉及 GPT-5.4,因此其较低的运行覆盖率部分反映了服务可靠性而非模型行为,但报告分数仍仅使用有效轨迹。其次,这为第 3.2 节中图 ?? 标注的预测偏差提供了运维层面的解释:由于第二半段可用性下降,GPT-5.4 的后期轨迹噪声更大,从而偏离基于前 6.5 小时的对数-S 形预测。

C 评测作弊

长视野评测给智能体许多适应评测反馈的机会。在任务构建与对抗性压力测试期间,我们审计了“在未真正调用目标能力的情况下提升测量分数”的策略。这些观察属于开发诊断,并非官方模型结果:受影响的任务被修订或排除,以下案例促使我们在最终基准中加入保护措施。

反馈作为隐藏答案的预言机。在 `cylinder_wake_prediction` 中,一个智能体将逐案例的绝对误差视为等式,通过超过 400 次提交重构了隐藏目标。形成的查找表在未经求解底层流体力学问题的情况下即评分为 1.000;而基于物理模型观察到的最佳提交仅得 0.165。

优化随机上尾。在 `nethack_dungeon_agent` 中,一个智能体在识别出更高方差会增加有利评测概率后,移除了固定随机种子。跨 311 次提交,其最佳得分为 1,501 而均值仅 484,展示了“最佳 N 选取”既奖励策略质量也奖励重复采样。

过拟合评测器种子。在 `bipedalwalker_locomotion_rl` 中,开发期判官最初使用了单一确定性 episode,使智能体能够推断并针对该被重用的种子进行优化。某次运行在判官种子上达到 Hardcore 回报 301.5,但在本地 100-episode 评测中仅约 12;另一次则直接针对判官公式搜索了小型手写控制器。这促使我们为随机性控制任务引入隐藏多种子评测。

跨越信任边界。在 `autolifter` 中,一个智能体发现反作弊检查豁免了仓库的 `baseline/` 目录,即将基于预言机的实现移动至该受信路径。它解决了隐藏 84 个用例中的 82 个并取得 0.980;遵循既定合成路线的最强观察提交仅得 0.121。

在线答案查找。公开索引的数据、任务制品或参考实现可能将既定推理问题退化为检索。在 `stock_momentum_bac` 中,一个智能体尝试对目标数据进行网络搜索,但任务的网络隔离阻断了请求。我们将此列为“已防止的风险”而非成功利用。

缓解措施。 这些发现促使我们同时强化任务设计与基础设施。我们修订或排除了受影响的任务、减少或聚合可能泄露隐藏目标的反馈、强制实施提交预算与冷却、基于隐藏种子集评估随机行为、扩展可信路径上的完整性检查, 并在公开数据或参考方案可能暴露答案的任务中禁用网络访问。这些控制不能消除每一种自适应攻击, 但减少了任务开发期间已识别的通道。

D 对数-S 形规律的完整推导

本附录给出观测到的“任务聚合环境学习曲线对数-S 形规律”的一种**充分机制**的完整推导。我们假设单个任务的得分通过有限多个可见的得分单元被观测, 因此其最佳-迄今曲线可以是带有长平台与突然突破的锯齿状阶梯。这些不规则性是单个任务在有限分辨率下的预期行为。在显式的切割混合、粒度、中点对齐与速度集中假设下, 我们展示了对许多独立评测的任务分数求平均如何能产生平滑的对数-S 形极限。

我们的核心建模假设: 环境学习是任务图上的前沿扩张过程。

我们将任务环境建模为一个**潜在图**, 其节点是小得分单元, 环境学习过程可被视为该图上的前沿扩张。我们的理论为“从前沿扩张过程到观测到的对数-S 形规律”提供了清晰路径。

推导的论述按五步展开:

1. **预备。** 定义模型条件化的任务图、得分单元、得分测度 μ 、影响矩阵 K 、其支撑图 E_K 与能力场。
2. **环境学习作为前沿扩张过程。** 模型根据先前收到的反馈改进其输出。在一个任务内部, 条件期望的得分增长率是任务图的“已解锁得分节点到锁定得分节点”的加权切割。
3. **单任务曲线在锯齿与平滑极限间变化。** 加权切割混合与小得分单元在多单元极限下将锯齿过程转化为 logistic 常微分方程。
4. **多任务聚合揭示对数-S 形规律。** 即使有限得分单元的扩张过程仍是锯齿过程, 基准聚合曲线收敛于平滑的对数-S 形规律。
5. **图自相似性诱导时间的对数尺度。** 当任务图在不同分辨率下显示自相似结构时, 前沿扩张自然呈现对数时间尺度。

贯穿本节, 我们用 $t > 0$ 表示原始交互时间, 原始任务/基准分数用 $S(t)$ 表示。设 $t_{\text{mid}} > 0$ 与 $S_{\text{max}} > 0$ 为拟合参数。对应的对数时间坐标与归一化分数为:

$$u = \log t - \log t_{\text{mid}}, \quad x(u) = \frac{S(t)}{S_{\text{max}}}.$$

我们展示了以下归一化对数-S 形动力学——其前沿速度为 β (也是拟合参数)——如何能作为已评测基准分数的多任务极限自然出现:

$$\frac{dx(u)}{du} = \beta x(u)(1 - x(u)), \quad x\left(\log \frac{t}{t_{\text{mid}}}\right) = \frac{1}{1 + (t_{\text{mid}}/t)^\beta}.$$

原始时间 $t = 0$ 处的值被理解为边界极限 $u \rightarrow -\infty$ 。

D.1 预备: 任务图与可达支撑

我们从一个任务与一个固定模型开始。核心假设是: 得分单元是一个“由智能体条件化的任务图”的节点; 已解锁节点提供影响场, 锁定节点通过有向影响边接收该场; 锁定节点可以以与场强成比例的速率变为已解锁。

定义 D.1 (任务图). 令 E 为任务的可见得分单元集合。语言模型诱导一个非负影响矩阵

$$K = (K_{ij})_{i,j \in E}, \quad K_{ij} \geq 0.$$

其有向支撑图的边集为 $E_K = \{j \rightarrow i : K_{ij} > 0\}$, 其中 $j \rightarrow i$ 表示已解锁源节点 j 可帮助解锁目标节点 i 。我们将 $G = (E, E_K)$ 记为由此得到任务图。

如果从一个更大的环境图开始, 对模型相关部分的限制在写出 E 与 K 之前就已进行; 经验上, 这种限制被吸收到拟合上限 $S_{\max}$ 中。接下来引入得分单元与对应的任务影响场。

定义 D.2 (得分单元与归一化得分测度). 在潜在图 $G = (E, E_K)$ 上, 每个节点 $i \in E$ 代表一个权重为 $\omega_i \geq 0$ 的得分单元。归一化得分测度为概率测度

$$W = \sum_{i \in E} \omega_i > 0, \quad \mu_i = \frac{\omega_i}{W}, \quad \mu(A) = \sum_{i \in A} \mu_i, \quad \forall A \subseteq E.$$

定义 D.3 (解锁状态, 已解锁集与锁定集). 节点 i 在对数时间 u 的解锁状态为 $n_i(u) \in \{0, 1\}$, 并且一旦节点被解锁, 便不能再次被锁定。归一化分数、已解锁集与锁定集定义为

$$x(u) = \sum_{i \in E} \mu_i n_i(u), \quad U(u) = \{j \in E : n_j(u) = 1\}, \quad L(u) = \{i \in E : n_i(u) = 0\}. \quad (5)$$

对静态状态 $n = (n_i)_{i \in E}$, 我们也记

$$U(n) = \{j \in E : n_j = 1\}, \quad L(n) = \{i \in E : n_i = 0\}, \quad x(n) = \mu(U(n)).$$

定义 D.4 (任务影响场). 元素 K_{ij} 是从源节点 j 到目标节点 i 的影响强度。目标节点 i 上的能力场为入边求和

$$h_i(n) = \sum_{j: j \rightarrow i} K_{ij} n_j = \sum_{j \in E} K_{ij} n_j.$$

在阐述多单元极限时, 我们恢复指标 N : 上述对象变为 $E_N, \mu^{(N)}, K^{(N)}, n^{(N)}(u), x_N(u), U_N(u), L_N(u)$, 以及

$$\mu_N(A) = \sum_{i \in A} \mu_i^{(N)}, \quad x_N(u) = \sum_{i \in E_N} \mu_i^{(N)} n_i^{(N)}(u), \quad h_i^{(N)}(u) = \sum_{j \in E_N} K_{ij}^{(N)} n_j^{(N)}(u).$$

对静态状态 $n \in \{0, 1\}^{E_N}$, 我们记

$$U_N(n) = \{j \in E_N : n_j = 1\}, \quad L_N(n) = \{i \in E_N : n_i = 0\}, \quad x_N(n) = \mu_N(U_N(n)).$$

D.2 环境学习作为前沿扩张过程

在潜在图与得分测度固定之后, 我们将故事转化为一个过程。使用前沿图景的原因在于, 长视野环境学习是累积性的。在软件任务中, 可运行的基线使后续调试有意义; 在证明任务中, 一个引理使后续证明义务更易; 在科学任务中, 经过校准的模型或验证循环使后续参数搜索更有用。一般而言, 每个部分成功都可能成为后续进展的工具。

图模型刻画了“已解决的得分单元可以如何帮助解锁剩余的”。在对数时间 u 时, 记

$$U(u) = \{j : n_j(u) = 1\}, \quad L(u) = \{i : n_i(u) = 0\}$$

为已解锁集与锁定集。锁定的单元 i 只能通过从 $U(u)$ 抵达的影响变为可用。因此, 相关量不是已解锁得分的总量, 而是从 $U(u)$ 进入 $L(u)$ 的影响量。

概率性前沿扩张。 对锁定的得分单元 i , 其任务场为 $h_i(u) = \sum_{j \in E} K_{ij} n_j(u)$ 。我们通过假设单元 i 以与累积场成比例的危险率被解锁, 将场的影响转化为连续时间随机过程:

$$\eta(1 - n_i(u))h_i(u) = \eta(1 - n_i(u)) \sum_{j \in E} K_{ij} n_j(u),$$

其中 $\eta > 0$ 将影响强度转换为每单位对数时间的进度。因此, 场不**确定性地**解锁 i ; 它决定了 i 被解锁的瞬时概率。这是“学习扩展前沿”的含义: 锁定节点以由来自已解锁一侧的当前影响决定的速率变为可用。我们在以下定义中形式化此过程。

定义 D.5 (单任务前沿过程). 固定有限任务图 $G = (E, E_K)$, 得分权重 μ_i , 影响矩阵 K 与转换常数 $\eta > 0$ 。单任务前沿过程是 $\{0, 1\}^E$ 上的连续时间过程, 其中对每个得分单元 i , 瞬时解锁强度为

$$\lambda_i(u) := \lim_{\Delta u \downarrow 0} \frac{\mathbb{P}(n_i(u + \Delta u) - n_i(u) = 1 \mid n(u))}{\Delta u} = \eta(1 - n_i(u)) \sum_{j \in E} K_{ij} n_j(u). \quad (6)$$

重要的是, 一旦节点被解锁, 它将保持解锁。

为表示由此产生的得分增长, 对 $A, B \subseteq E$ 定义加权前沿切割

$$C(A, B) = \sum_{i \in A} \sum_{j \in B} \mu_i K_{ij}.$$

这是从源集 B 到目标集 A 的总影响, 每个目标按若解锁所获得分加权。时间 u 的活动前沿因此为 $C(L(u), U(u))$ 。

引理 D.1 (单任务前沿精确增长率). 对单任务前沿过程,

$$\frac{d}{du} \mathbb{E}[x(u) \mid n(u)] = \eta C(L(u), U(u)).$$

证明. 固定 u 并以状态 $n(u) = n$ 为条件。对 $i \in E$, 令 $A_i(\Delta u)$ 表示节点 i 在 $[u, u + \Delta u]$ 内解锁的事件。由解锁强度的定义,

$$\mathbb{P}(A_i(\Delta u) \mid n(u) = n) = \eta(1 - n_i) \sum_{j \in E} K_{ij} n_j \Delta u + o(\Delta u).$$

由于 E 有限, 在 $[u, u + \Delta u]$ 内发生两次或更多解锁事件的概率为 $o(\Delta u)$ 。因此, Δu 的一阶项下的期望得分增量通过对每个可能的单节点解锁求和得到:

$$\mathbb{E}[x(u + \Delta u) - x(u) \mid n(u) = n] = \sum_{i \in E} \mu_i \mathbb{P}(A_i(\Delta u) \mid n(u) = n) + o(\Delta u) = \eta \sum_{i \in E} \sum_{j \in E} \mu_i (1 - n_i) K_{ij} n_j \Delta u + o(\Delta u).$$

除以 Δu 并令 $\Delta u \downarrow 0$ 得

$$\frac{d}{du} \mathbb{E}[x(u) \mid n(u) = n] = \eta \sum_{i \in E} \sum_{j \in E} \mu_i (1 - n_i) K_{ij} n_j.$$

由于 $1 - n_i$ 将外侧求和限制到 $i \in L(n)$, n_j 将内侧求和限制到 $j \in U(n)$, 故有

$$\frac{d}{du} \mathbb{E}[x(u) \mid n(u) = n] = \eta \sum_{i \in L(n)} \sum_{j \in U(n)} \mu_i K_{ij} = \eta C(L(n), U(n)).$$

□

引理 ?? 是关键简化。它表明, 为了获得 logistic 期望增长率, 我们不需要每条微观边权重都相同; 而仅需“边界影响”主要依赖于两个粗粒度量: 已解锁质量 $\mu(U)$ 与锁定质量 $\mu(L)$ 。则场大致为这两者的乘积: 可用的可复用能力乘以剩余的分数机会。

随着得分单元数量增加, 固定分辨率下的相同对象现在被应用于规模为 N 的图。特别地,

$$C_N(A, B) = \sum_{i \in A} \sum_{j \in B} \mu_i^{(N)} K_{ij}^{(N)}, \quad A, B \subseteq E_N.$$

从前沿切割到乘积前沿。剩余的建模问题是: 何时该边界影响看起来像两个测度的乘积? 以下量控制前沿过程与完全混合情形的偏差。

定义 D.6 (单任务加权切割误差). 给定 $(E_N, \mu^{(N)}, K^{(N)})$ 与任意 $\kappa > 0$, 定义

$$\varepsilon_N(\kappa) = \sup_{A, B \subseteq E_N} \left| \sum_{i \in A} \sum_{j \in B} \mu_i^{(N)} (K_{ij}^{(N)} - \kappa \mu_j^{(N)}) \right|.$$

条件 D.1 (单任务加权切割混合). 存在 $\kappa > 0$ 使得, 对任务图, 当 $N \rightarrow \infty$ 时, 定义 D.6 中的加权切割误差满足

$$\varepsilon_N \rightarrow 0.$$

加权切割混合弱于“逐项完全混合”。它不是说每个 $K_{ij}^{(N)}$ 都接近 $\kappa \mu_j^{(N)}$, 而是说, 对每个可能的前沿, 从已解锁图节点 j 进入已锁定图节点的“聚合能力”接近乘积测度值。

引理 D.2 (切割混合给出 logistic 前沿增长率). 在条件 D.1 下, 以下对每个静态状态 $n \in \{0, 1\}^{E_N}$ 成立。记 $U_N = U_N(n)$, $L_N = L_N(n)$, $x_N = x_N(n)$ 。则

$$|C_N(L_N, U_N) - \kappa x_N(1 - x_N)| \leq \varepsilon_N.$$

因此,

$$b_N(n) = \eta \kappa x_N(1 - x_N) + r_N(n), \quad |r_N(n)| \leq \eta \varepsilon_N.$$

证明. 将定义 D.6 应用到 $A = L_N$, $B = U_N$ 。由于 $\mu_N(U_N) = x_N$ 且 $\mu_N(L_N) = 1 - x_N$, 故有

$$|C_N(L_N, U_N) - \kappa x_N(1 - x_N)| = \left| \sum_{i \in L_N} \sum_{j \in U_N} \mu_i^{(N)} (K_{ij}^{(N)} - \kappa \mu_j^{(N)}) \right| \leq \varepsilon_N.$$

期望增长率的陈述由引理 D.1 得出。 □

切割混合控制了确定性期望增长率。单独的条件控制可见的跳跃大小。这一区分对本文的解读至关重要: 仅有少量大得分单元的有限任务可以遵循正确的前沿机制, 却仍表现得非常锯齿。

条件 D.2 (小得分单元与受控场). 设

$$q_N = \sum_{i \in E_N} (\mu_i^{(N)})^2.$$

假设 $q_N \rightarrow 0$ 。还假设存在确定性 H_N 使得

$$\sup_{i \in E_N, n \in \{0, 1\}^{E_N}} \sum_{j \in E_N} K_{ij}^{(N)} n_j \leq H_N, \quad H_N q_N \rightarrow 0.$$

对等大得分单元, $q_N = 1/N$ 。因此条件 D.2 捕捉了从“任务具有少量大得分单元”到“任务具有许多小得分单元”的平滑化过程。下面的定理应以这种方式解读。它并非声称有限基准任务的实现最佳-迄今曲线必须在视觉上平滑; 而是说, 若同一任务级机制在越来越细的得分分辨率下被观测, 则阶梯过程将收敛到 logistic 前沿。

定理 D.1 (单任务前沿的多单元极限). 考虑定义 D.5 中的单任务前沿过程。设 $[u_0, u_1]$ 是一个紧的对数时间区间, $x_0 \in [0, 1]$ 。假设条件 D.1 与 D.2 成立, 且 $x_N(u_0) \rightarrow x_0$ 。则 x_N 在 $[u_0, u_1]$ 上依概率一致收敛到

$$\frac{dx}{du} = \eta\kappa x(1-x), \quad x(u_0) = x_0, \quad (7)$$

的解。若选取 t_{mid} 使极限中点满足 $x(0) = 1/2$, 则令 $x(t) := x(\log(t/t_{\text{mid}}))$ 即得对数-S 形规律:

$$\frac{1}{x_N} \xrightarrow{P} x(t) = \frac{1}{1 + (t_{\text{mid}}/t)^\beta}, \quad \beta = \eta\kappa. \quad (8)$$

证明. 令 $M_N(u)$ 表示 Doob-Meyer 分解中的鞅增量, 归一化使得 $M_N(u_0) = 0$ 。Doob-Meyer 分解与引理 D.2 给出, 对 $u \in [u_0, u_1]$,

$$x_N(u) = x_N(u_0) + M_N(u) + \eta\kappa \int_{u_0}^u x_N(s)(1-x_N(s)) ds + R_N(u), \quad (9)$$

其中

$$\sup_{u \in [u_0, u_1]} |R_N(u)| \leq \eta(u_1 - u_0)\varepsilon_N.$$

节点 i 的单次跳跃将 x 改变 $\mu_i^{(N)}$ 。在 $[u_0, u_1]$ 上累积的可料二次变差因此以

$$\langle M_N \rangle_{[u_0, u_1]} = \int_{u_0}^{u_1} \sum_{i \in E_N} (\mu_i^{(N)})^2 \eta(1 - n_i^{(N)}(s)) \sum_{j \in E_N} K_{ij}^{(N)} n_j^{(N)}(s) ds \leq \eta(u_1 - u_0) H_N q_N$$

为界。Doob L^2 不等式因此给出

$$\mathbb{E} \sup_{u \in [u_0, u_1]} |M_N(u)|^2 \leq 4\eta(u_1 - u_0) H_N q_N \rightarrow 0.$$

因此鞅项依概率一致地趋于零。令 $f(x) = \eta\kappa x(1-x)$ 。在 $[0, 1]$ 上, f 是 Lipschitz 的, 常数不超过 $\eta\kappa$ 。将 (9) 与 (7) 的积分方程比较, 得

$$\sup_{u_0 \leq r \leq u} |x_N(r) - x(r)| \leq |x_N(u_0) - x_0| + \sup_{u_0 \leq r \leq u} |M_N(r)| + \eta(u_1 - u_0)\varepsilon_N + \eta\kappa \int_{u_0}^u \sup_{u_0 \leq r \leq s} |x_N(r) - x(r)| ds.$$

Gronwall 不等式证明在 $[u_0, u_1]$ 上依概率一致收敛。解 (7) 并设 $x(0) = 1/2$ 即得 (8)。□

解读单任务曲线形状。 对有限 N , 任务分数仍然是锯齿跳跃过程。该定理是说, 平滑 logistic 曲线仅在两个效果可忽略后出现: 混合切割误差 $\varepsilon_N \rightarrow 0$ 以及来自大得分单元的跳跃噪声 $H_N q_N \rightarrow 0$ 。因此, 锯齿的有限任务曲线与 logistic 前沿机制相容。此外, 增长率 $x(1-x)$ 有直接解读: 已解锁得分测度 x 提供能力以帮助解锁新得分, 而锁定得分测度 $1-x$ 界定了分数改进的剩余机会。在下小节中, 我们将解释为何对许多这样的有限任务曲线求平均是预期得到简洁缩放规律的合理场所。

D.3 多任务聚合揭示平滑的对数-S 形规律

我们现在回到基准中拟合的量: 跨许多独立评测任务的平均。如前所见, 逐任务曲线是异构且常常可见地锯齿的; 而跨任务均值曲线则平滑得多, 且随着包含更多任务, 越来越被对数-S 形良好拟合。在我们的语言里, 每个任务都是任务图上其自身的有限前沿扩张过程。聚合定理因此回答了核心问题:

许多锯齿的任务曲线如何能在任务求平均后产生简洁的对数-S 形缩放规律?

答案有两层。首先, 由于不同任务环境互不交互, 每个任务可以具有其自身的近似 logistic 前沿。其次, 基准求平均消除了有限任务的粗糙, 当任务中点与任务速度足够集中时, 聚合变成单一的对数-S 形。

从单一任务到完整基准引入了三层额外的变异性:

- **任务级 logistic 前沿动力学。** 每个任务都有其自身的任务图、得分原子、影响矩阵、切割误差与有限跳跃噪声。这些量决定了该任务的前沿扩张在多大程度上遵循其 logistic 极限。
- **时间轴对齐。** 每个任务可能有其模型相关的中点 $t_{\text{mid},b}$ 。然而, 基准级拟合只有一个公共中点 t_{mid} 。因此, 任务特定的中点位移必须在跨任务间集中。
- **学习速度同质性。** 每个任务也可能有其自身的前沿速度 β_b 。基准级拟合使用一个公共速度 β , 因此模型相关的任务学习速度必须围绕一个共享值集中。

聚合定理表明, 对数-S 形规律作为总体级极限出现: 有限任务粗糙被求平均冲刷, 当其中点与速度充分对齐时, 剩余的任务级 logistic 前沿组合为单一曲线。

定义 D.7 (基准任务图与状态). 对含 M 个任务的基准, 任务由 $b = 1, \dots, M$ 索引。任务 b 有其自身的任务图

$$G_b = (E_b, E_b), \quad E_b = \{j \rightarrow i : K_{ij}^b > 0\},$$

其中 E_b 是该任务对被评测模型的可见得分单元集。它具有归一化得分测度 μ_i^b (对 $i \in E_b$)、影响矩阵 $K^b = (K_{ij}^b)$ 、以及场到进度的转换常数 $\eta_b > 0$ 。任务 b 的状态向量为

$$n^b(u) = (n_i^b(u))_{i \in E_b},$$

节点 i 上的能力场为

$$h_i^b(u) = \sum_{j \in E_b} K_{ij}^b n_j^b(u).$$

定义 D.8 (任务分数与基准均值). 任务 b 在对数时间 u 的归一化分数为

$$x_b(u) = \sum_{i \in E_b} \mu_i^b n_i^b(u).$$

而基准均值的归一化分数为

$$x_B(u) = \frac{1}{M} \sum_{b=1}^M x_b(u).$$

这里 $u = \log t - \log t_{\text{mid}}$ 为对数时间坐标。对原始时间, 我们用

$$x_B^\dagger(t) = x_B\left(\log \frac{t}{t_{\text{mid}}}\right), \quad S_B(t) = S_{\text{max}} \cdot x_B^\dagger(t),$$

其中 t_{mid} 为拟合的聚合中点, S_{max} 为拟合的聚合上限。个体任务可能相对于此公共坐标存在残余中点位移, 如下文 (10) 中引入的 δ_b 。

定义 D.9 (任务特定 logistic 前沿). 在基准级对数时间坐标 u 中, 任务 b 被赋予任务特定的前沿速度 $\beta_b \geq 0$ 与残余中点位移 $\delta_b \in \mathbb{R}$ 。对应的任务特定 logistic 曲线为

$$\ell_b(u) = \frac{1}{1 + e^{-\beta_b(u-\delta_b)}}. \quad (10)$$

这里 β_b 控制任务级前沿传播的速度, 而 δ_b 度量任务中点相对于基准级中点 t_{mid} 。因此 $\delta_b = 0$ 意味着任务 b 的中点与聚合中点对齐。

第一个条件将单任务切割混合论证应用于每个任务图内。它针对影响矩阵 $\eta_b K^b$ 写出, 因此乘积前沿系数直接是 β_b 。

定义 D.10 (逐块加权切割误差与场上界). 对每个任务 b , 通过

$$\varepsilon_b = \sup_{A, B \subseteq E_b} \left| \sum_{i \in A} \sum_{j \in B} \mu_i^b (\eta_b K_{ij}^b - \beta_b \mu_j^b) \right| \quad (11)$$

定义任务条件化加权切割误差。通过

$$q_b = \sum_{i \in E_b} (\mu_i^b)^2, \quad H_b = \sup_{i \in E_b, n \in \{0,1\}^{E_b}} \sum_{j \in E_b} \eta_b K_{ij}^b n_j$$

定义块粒度与场上界。

条件 D.3 (逐块加权切割混合与消失的得分单元). 假设任务速度一致有界,

$$0 \leq \beta_b \leq \beta_+ < \infty,$$

且平均任务切割误差与粒度误差消失:

$$\Delta = \frac{1}{M} \sum_{b=1}^M \varepsilon_b \rightarrow 0, \quad A = \frac{1}{M} \sum_{b=1}^M \sqrt{H_b q_b} \rightarrow 0 \quad (12)$$

依概率。

下一个条件要求不同任务的初始值变化可忽略。

条件 D.4 (逐块初始对齐). 我们假设动力学的平均初值对齐:

$$\frac{1}{M} \sum_{b=1}^M |x_b(u_0) - \ell_b(u_0)| \xrightarrow{P} 0.$$

条件 D.3 有一个直接推论: 观测到的基准轨迹接近任务特定 logistic 前沿的平均。注意条件中的求平均。我们并不要求每个任务在其自身看起来平滑, 但平均的切割误差、得分单元与跳跃贡献必须消失。这是“一群锯齿的前沿扩张过程能够揭示平滑规律”这一经验性主张的正式版本。对紧区间 $I \subseteq \mathbb{R}$, 定义以下逐块轨迹误差上确界:

$$R_M(I) := \sup_{u \in I} \left| x_B(u) - \frac{1}{M} \sum_{b=1}^M \ell_b(u) \right| = \sup_{u \in I} \left| \frac{1}{M} \sum_{b=1}^M (x_b(u) - \ell_b(u)) \right|. \quad (13)$$

我们有下述引理证明其消失性。

引理 D.3 (平均误差上确界的多任务极限). 在条件 D.3 与 D.4 下, 对任何紧区间 $I \subseteq \mathbb{R}$, $R_M(I) \xrightarrow{P} 0$ 当 $M \rightarrow \infty$ 。

证明. 对任务 b , 与单任务证明相同的 Doob–Meyer 分解给出

$$x_b(u) = x_b(u_0) + M_b(u) + \int_{u_0}^u \{\beta_b x_b(s)(1 - x_b(s)) + \rho_b(s)\} ds, \quad (14)$$

其中 M_b 是鞅, 且由 (11), 对所有状态 $|rho_b(s)| \leq \varepsilon_b$. 任务级曲线 (10) 满足

$$\frac{d\ell_b}{du} = \beta_b \ell_b (1 - \ell_b).$$

由于 $z \mapsto \beta_b z(1 - z)$ 在 $[0, 1]$ 上 Lipschitz, 常数不超过 β_+ , 将 Gronwall 不等式应用于 (14) 得

$$\sup_{u \in I} |x_b(u) - \ell_b(u)| \leq e^{\beta_+ |I|} (|x_b(u_0) - \ell_b(u_0)| + \sup_{u \in I} |M_b(u)| + |I| \varepsilon_b), \quad (15)$$

其中 $|I| = u_1 - u_0$. 第一项依条件 D.4 收敛到零, 第三项依条件 D.3 收敛到零。剩下对鞅项求平均。任务 b 中单元 i 的跳跃将 x 改变 μ_i^b , 且总得分增长场上界为 H_b . 因此 M_b 的可料二次变差可被界定为

$$\langle M_b \rangle_I \leq |I| H_b q_b.$$

Doob L^2 不等式给出条件性界

$$\mathbb{E} \left[\sup_{u \in I} |M_b(u)| \middle| G_M \right] \leq 2 \sqrt{|I| H_b q_b}, \quad (16)$$

其中 G_M 表示任务级权重与影响矩阵。这种条件形式是有用的, 因为 A_M 自身可能是随机的。令

$$Z_M = \frac{1}{M} \sum_{b=1}^M \sup_{u \in I} |M_b(u)|.$$

对任意 $\varepsilon > 0$ 与 $a > 0$, 由 (16) 得

$$\mathbb{P}(Z_M > \varepsilon) \leq \mathbb{P}(A_M > a) + \mathbb{P}(Z_M > \varepsilon, A_M \leq a) \leq \mathbb{P}(A_M > a) + \frac{2\sqrt{|I|a}}{\varepsilon}.$$

由于 $A_M \xrightarrow{P} 0$, 先令 $M \rightarrow \infty$, 再令 $a \downarrow 0$ 即得 $Z_M \xrightarrow{P} 0$. 对 (15) 关于 b 求平均并使用 (12) 即证 (13). $\square$

接下来的两个任务分布假设用于在基准内对齐缩放曲线。它们并非逐块切割混合的推论。它们是将任务级 S 形的平均转化为单一基准 S 形的关键所在。

假设 D.1 (一致对数时间偏置). 存在单一基准级中点 t_{mid} 使得 (10) 中的残余任务位移满足

$$D_M = \frac{1}{M} \sum_{b=1}^M |\delta_b| \rightarrow 0.$$

严格一致偏置情形为 $\delta_b \equiv 0$.

假设 D.2 (集中的环境学习速度). 存在标量 $\beta > 0$ 使得

$$B_M = \frac{1}{M} \sum_{b=1}^M |\beta_b - \beta| \rightarrow 0.$$

特别地, $M^{-1} \sum_{b=1}^M \beta_b \rightarrow \beta$.

可以从前沿聚合中看出这些假设为何必要。求平均消除了锯齿性, 但并未自行对齐任务难度或环境学习速度。当逐块切割混合误差趋于零时, 主项期望增长率为

$$\frac{1}{M} \sum_{b=1}^M \beta_b x_b(u)(1 - x_b(u)).$$

当且仅当任务被充分对齐, 使得任务分数与速度表现得像一个总体时, 这才变成 $\beta x_B(1 - x_B)$ 。即使当所有速度相等时, 持续的任务离散会通过下式从标量前沿中减去

$$\frac{1}{M} \sum_{b=1}^M x_b(1 - x_b) = x_B(1 - x_B) - \frac{1}{M} \sum_{b=1}^M (x_b - x_B)^2.$$

下面的定理证明, 一旦逐块前沿条件、中点对齐与速度集中就位, 核心收敛性即成立。

定理 D.2 (基准聚合在多任务极限下产生对数-S 形规律). 固定紧对数时间区间 $I \subseteq \mathbb{R}$ 。假设条件 D.3–D.4、假设 D.1–D.2 对某个 $t_{\text{mid}} > 0$ 与 $\beta > 0$ 成立。则

$$x_B(u) \xrightarrow{P} \ell_\beta(u), \quad \forall u \in I.$$

等价地, 对原始时间 t 的对数坐标 $u = \log(t/t_{\text{mid}})$ 位于 I 中者, 使用定义 D.8 中的记号, 我们有

$$x_B^\dagger(t) = \frac{S_B(t)}{S_{\max}} \xrightarrow{P} \frac{1}{1 + (t_{\text{mid}}/t)^\beta}. \quad (17)$$

证明. 我们对归一化分数 $x_B(u)$ 证明该陈述。乘以固定拟合上限 $S_{\max}$ 给出原始分数的陈述。

步骤 1: 用任务级前沿极限替换观测到的任务轨迹。由引理 D.3,

$$\sup_{u \in I} \left| x_B(u) - \frac{1}{M} \sum_{b=1}^M \ell_b(u) \right| \leq \sup_{u \in I} \left| \frac{1}{M} \sum_{b=1}^M (x_b(u) - \ell_b(u)) \right| = R_M(I) \rightarrow 0$$

依概率。

步骤 2: 将任务级前沿对齐到基准前沿。由于 logistic 链接满足 $|\text{sigmoid}'(z)| \leq 1/4$ 对所有 z , 记 $\ell_b(u) = \text{sigmoid}(\beta_b u)$ 。我们有

$$\sup_{u \in I} |\ell_b(u) - \ell_\beta(u)| \leq \frac{1}{4} \sup_{u \in I} |\beta_b(u - \delta_b) - \beta u| \leq \frac{1}{4} \sup_{u \in I} |u| |\beta_b - \beta| + \beta_b |\delta_b|.$$

条件 D.3 中的一致速度界给出 $\beta_b |\delta_b| \leq \beta_+ |\delta_b|$ 。对任务求平均并使用假设 D.1 与 D.2,

$$\frac{1}{M} \sum_{b=1}^M \sup_{u \in I} |\ell_b(u) - \ell_\beta(u)| \leq \frac{1}{4} \sup_{u \in I} |u| B_M + \beta_+ D_M \rightarrow 0.$$

步骤 3: 组合两个近似。三角不等式给出

$$\sup_{u \in I} |x_B(u) - \ell_\beta(u)| \leq \sup_{u \in I} \left| x_B(u) - \frac{1}{M} \sum_{b=1}^M \ell_b(u) \right| + \frac{1}{M} \sum_{b=1}^M \sup_{u \in I} |\ell_b(u) - \ell_\beta(u)|.$$

第一项依 (18) 收敛到零, 第二项依 (20) 收敛到零。这证明 x_B 在 I 上依概率一致收敛到 ℓ_β 。代入 $u = \log(t/t_{\text{mid}})$ 即得 (17)。□

基准聚合如何产生经验缩放规律。 该定理对应于经验缩放规律。聚合平均了许多任务图上的前沿扩张过程以产生在多任务极限下的涌现平滑曲线。为使收敛成立, 我们需要逐块切割混合以保证任务级乘积前沿增长率, 需要小得分单元条件以使聚合中的跳跃噪声消失, 以及需要中点与速度假设以防止混合不兼容的 S 形。在这些条件下, 基准均值最终可在 (17) 中收敛为单一对数-S 形规律, 而非仅仅呈 S 形。

D.4 图自相似性诱导时间的对数尺度

前几节解释了为何一旦进度以有效的“学习坐标”度量, 前沿扩张便给出 logistic 动力学。现在我们解释为何该坐标常常应是原始交互时间的对数。

回想前沿过程已发生在模型条件化任务图 $G = (E, E_K)$ 上。边 $j \rightarrow i$ 存在意味着已解锁源节点 j 可帮助解锁目标节点 i , 场强为 K_{ij} 。加权切割 $C(L, U)$ 度量从已解锁集 U 到锁定集 L 的总影响。

剩余的问题是: 原始交互时间如何驱动智能体穿越潜在任务图。我们提出的答案是**图相关且几何的**: 任务图的更困难区域需要更多搜索努力才能暴露。如果该搜索几何在难度尺度上近似**自相似**, 那么难度的每一**加法**增加需要原始时间的**乘法**增加。这正是自然的前沿坐标变为 $\log t$ 的原因。

定义 D.11 (任务图边上的难度尺度). 对每个影响边 $j \rightarrow i \in E_K$, 令 $\rho_{ij} \geq 0$ 为该边的难度尺度。较小的 ρ 意味着 j 到 i 的影响在较低搜索努力下即可用; 较大的 ρ 意味着该影响需要更深搜索或更长交互才可用。对 $r \geq 0$, 定义已暴露的影响矩阵

$$K_{ij}^{(\leq r)} = \begin{cases} K_{ij}, & \text{若 } j \rightarrow i \in E_K \text{ 且 } \rho_{ij} \leq r, \\ 0, & \text{否则.} \end{cases}$$

对应的已暴露场为

$$h_i^{(\leq r)}(n) = \sum_{j \in E} K_{ij}^{(\leq r)} n_j.$$

增大 r 揭示了同一任务图的更多部分。在小 r 处, 仅低难度边对场有贡献; 在较大 r 处, 更高难度的边也对前沿切割有贡献。因此 r 是关于“多少任务图已变得可用”的有效坐标。

定义 D.12 (难度尺度下的搜索量). 令 $k \in \mathbb{N}_+$ 索引难度等级, 等级 k 对应尺度带 $[k\Delta r, (k+1)\Delta r)$, $\Delta r > 0$ 。定义等级 k 下的任务图边集为

$$E_k = \{j \rightarrow i \in E_K : k\Delta r \leq \rho_{ij} < (k+1)\Delta r\}.$$

设 $N_k = |E_k|$ 为该尺度带中边数。我们将 N_k 解读为搜索量的离散代理: 它计数了在难度等级 k 下必须暴露多少边结构。

搜索量与得分测度不同。得分测度 μ 度量在节点被解锁后获得多少基准价值。搜索量度量在相应的前沿影响出现之前必须变得可用的任务图结构量。

假设 D.3 (自相似任务图边增长). 存在常数 $b > 1$ 与 $c_-, c_+ > 0$ 使得, 在感兴趣的尺度范围内,

$$c_- b^k \leq N_k \leq c_+ b^k.$$

该假设意味着, 难度等级的每一加法增加将相关任务图边结构量乘以一个可与 b 比较的因子。由于等级 k 对应尺度 $r_k = k\Delta r$, 这意味着

$$N_k \asymp b^{r_k/\Delta r} = \exp\left(\frac{\log b}{\Delta r} r_k\right).$$

我们记

$$h = \frac{\log b}{\Delta r}$$

为对应的搜索熵参数。直观地, h 度量任务图结构量随难度尺度增加而增长的速度。

离散假设激励出连续近似

$$V(r) = V_0 e^{hr}, \quad V_0 > 0,$$

其中 $V(r)dr$ 是暴露难度带 $[r, r + dr]$ 中任务图边结构所需的搜索努力。因此, 到难度尺度 r 的累积搜索量为

$$V(r) = \int_0^r V(s)ds = \frac{V_0}{h}(e^{hr} - 1).$$

假设 D.4 (线性搜索努力供给). 令 $A(t)$ 为原始交互时间 t 提供的累积搜索努力。假设

$$A(t) = \nu t$$

对某 $\nu > 0$ 。

该假设意味着原始交互时间与可用搜索努力成正比。搜索可以是自适应的, 但一单位原始时间本身不会暴露任务图指数多的边。

命题 D.1 (自相似任务几何给出对数时间). 在假设 D.3 与 D.4 下, 原始时间 t 暴露的难度尺度满足

$$r(t) = \frac{1}{h} \log \left(1 + \frac{h\nu}{V_0} t \right).$$

特别地, 在无标度区 $t \gg V_0/(h\nu)$ 中,

$$r(t) = \frac{1}{h} \log t + O(1).$$

该命题只是累积搜索量的求逆。尺度 $r(t)$ 由 $V(r(t)) = A(t)$ 定义。由于 $V(r)$ 指数增长于 r , 而 $A(t)$ 线性增长于 t , 暴露的难度尺度对数增长于原始时间。

与前沿规律的联系。 前述前沿规律刻画了“一旦进度以使任务图影响可用的坐标度量时”的得分增长。此处该坐标是 r 。随 r 增加, K 的更多元素通过 $K^{(\leq r)}$ 被暴露, 场 $h^{(\leq r)}$ 增大, 前沿切割从未解锁节点到锁定节点通过与之前相同的机制扩张。如果已暴露的任务图在每个尺度上近似加权切割混合, 且具有尺度平稳的有效前沿系数, 则尺度坐标中的粗粒前沿动力学取乘积形式

$$\frac{dx}{dr} = \gamma x(1 - x),$$

其中 $\gamma > 0$ 是每单位难度尺度的前沿速度。这是之前的加权切割前沿机制写在 r 坐标中的版本。

由于在无标度区 $r(t) = h^{-1} \log t + O(1)$, 对数时间每增加一单位对应于难度尺度的约 $1/h$ 单位进度。因此

$$\frac{dx}{d \log t} = \frac{\gamma}{h} x(1 - x).$$

于是拟合的对数时间前沿速度为

$$\beta = \frac{\gamma}{h}.$$

求解对数时间 logistic 方程即得

$$x(t) = \frac{1}{1 + (t_{\text{mid}}/t)^\beta}.$$

自相似任务几何为何给出对数时间。 加权切割论证解释了前沿因子 $x(1 - x)$, 如前所示。自相似任务图几何则解释了“时间坐标应以对数尺度度量”的原因。如果难度尺度的每一加法增加将相关任务图边数乘以因子 b , 则搜索量以 e^{hr} 增长, 其中 $h = (\log b)/\Delta r$ 。由于原始交互时间仅提供 $O(t)$ 的搜索努力, 暴露的难度尺度满足 $r(t) = h^{-1} \log t + O(1)$ 。因此, 任务图尺度上的 logistic 前沿律变为 $\log t$ 上的 logistic 律。

D.5 讨论与局限

上述推导给出了对数-S 形经验规律的一种充分机制。其假设之所以有用, 正是在于它们指明了该对数-S 形极限可能失败的具体方式。因此, 我们将其视为对所观察到的规律的一种**机制性说明**, 而非声称所有环境学习曲线都必须是 logistic 的。

有限得分粒度。 单任务理论允许有限任务保持视觉上锯齿。现实任务可能含有少量大的隐藏测试、决定性的证明义务, 或高权重的评分单元。当这些得分单元保持宏观大小时, 鞅误差项不必对该任务消失, 而实现最佳-迄今曲线可呈现长平台与突然跳跃。聚合定理仅要求此类粗单元不主导基准均值。若基准得分测量的不可忽略部分由粗任务承载, 则即便平均漂移近似 logistic, 聚合曲线仍可能保留可见的跳跃或较大的运行间方差。

加权切割混合。 加权切割混合是缩放规律的核心条件。它表明每个宏观的已解锁-锁定前沿切割都看到近似乘积测度的影响。如果任务图具有持续的瓶颈、模块、先决链, 或分离开的高转移与低转移区域, 则前沿记忆的是“其在图中的位置”, 而非仅“已解锁了多少得分测度”。在此情形下, 自然极限不再是一维 logistic 方程。我们应转而预期多类型动力学、延迟起飞、多重拐点、长平台, 或对应于不同任务模块的多个 S 形之和。

可达支撑与拟合上限。 该分析将 $S_{\max}$ 视为可有效到达的得分测度的稳定支撑。当实际可达到的得分单元集在拟合时间窗口上固定时, 这是合适的。然而, 若更长交互改变了什么可有效到达——例如弱转移路径仅在更长视野上变得可用——则归一化分数的分母本身在移动。短窗口拟合仍可提供有用的有效上限, 但该上限不应被解读为无期限上界。

任务中点对齐。 聚合定理假设单一基准级中点能去除大部分任务级对数时间偏置。若残余中点位移 δ_b 仍广泛分散, 则基准均值变为位移任务前沿的卷积。这种曲线仍可能平滑且 S 形, 但不必满足标量 logistic ODE。在此情形下, 拟合的中点与斜率是任务中点分布的窗口依赖性总结, 而非内在的基准常数。

学习速度集中。 聚合定理还假设任务前沿速度集中于一个公共值。若某些任务家族持续具有陡前沿, 另一些则具有持续浅前沿, 则任务级 S 形的平均通常不再自身是 S 形。早期进度可能由快任务主导, 而后期进度由慢任务主导。单一拟合 β 因此可能反映任务家族组成而非真正的标量环境学习速度。

时间坐标的选择。 对数时间坐标由图自相似性假设论证: 难度每加法增加需要搜索努力的乘法增加。然而, 某些环境具有特征的原始时间周期: 固定评测延迟、每日数据刷新、硬截止、分阶段课程或批量反馈。在这些情形下, 另一种时间坐标或分段时间模型可能比单一对数时间变换更合适。

综上, 这些局限阐明了证明的范围。本附录给出了“从前沿扩张到观测到的总体级对数-S 形规律”的一条路径。一种更完整的理论将分类由粗得分单元、非混合图、动态可达支撑、分散中点、异构学习速度以及非自相似反馈调度所产生的非 logistic 极限。

E 缩放规律形状的更多讨论

机制性解读选择对数-S 形。由于候选 S 形拟合得几乎一样好 (表 1; 对数-S 形与对数-Probit 的简并性自 Berkson [?] 以来便为人知), 选择不能基于拟合优劣; 我们基于机制做出。将候选 S 形写为归一化分数 $y = S/S_{\max}$ 的增长律, 每个候选对“什么驱动进度, 什么限制它”做出了不同陈述。

- **对数-S 形:** $\frac{dy}{dt} = \beta y(1 - y)$, 其中 $\tau = \ln t$ 。详细推导见第 3.3 节与附录 D; 此处关键机制性解读是 y 是已解锁的可达得分质量, $1 - y$ 是剩余锁定质量。如果跨已解锁-锁定前沿的聚合影响与这些质量的乘积近似成正比, 则进度遵循 $y(1 - y)$ 。对数时间坐标源自任务图的自相似搜索几何。该解读与两项经验检验一致: 第 5.2 节显示连续有状态运行优于等预算的重复采样, 所以进度不能仅由独立尝试解释; 第 5.4 节显示一个有限任务通过稀疏但累积的突破取得进展, 早期可行流水线使后续修复可被搜索。拐点位于 $y = 0.5$ (对称)。
- **对数-Gompertz:** $\frac{dy}{dt} = c y \ln(1/y)$ 。 $\ln(1/y)$ 项是“随系统成熟, 引擎以乘法方式减速”的标志 (例如肿瘤增长, 其中增殖分数缩小)。其进度前重, 拐点位于 $y = 1/e \approx 0.37$: 相对增长率在 $y \rightarrow 0$ 时发散, 因此一个微小系统因为它最终将遇到的限制尚不存在而爆炸式增长。经验获取则相反——其早期阶段缓慢, 因为必须先建立立足点, 而非因缺少刹车而迅速。
- **对数-Probit:** $\frac{dy}{dt} = \frac{1}{\sigma} \phi(\Phi^{-1}(y))$, 跨对数正态难度的扫描——这些难度由许多独立因子的乘积形成 (乘法中心极限定理)。经验获取是路径依赖的而非独立因子的乘积, 因此该微观基础并不适用, 即使 probit 与 logistic 在经验上几乎不可区分。
- **Weibull CDF:** $y(t) = 1 - \exp\{-(t/\lambda)^\beta\}$, 等价地 $\frac{dy}{dt} = h(t)(1 - y)$, 其中 $h(t) = \frac{\beta}{\lambda} (t/\lambda)^{\beta-1}$ 。其危险率是原始已用时间的函数, 而累积进度仅通过生存项 $(1 - y)$ 进入。这使其成为自然的首达或重复采样基线。对单 0-1 奖励任务、独立尝试的成功概率为 p , 有 $\mathbb{P}(\text{pass at } k) = 1 - (1 - p)^k = 1 - \exp[-k(-\ln(1 - p))]$, 即 $\beta = 1$ 的 Weibull 情形 (对 p 小时近似 $1 - \exp(-pk)$)。这种曲线之所以改进, 是因为重复独立尝试缩小了剩余失败质量, 而非因为跨尝试传递的状态使后续尝试更有效。第 5.2 节显示有状态学习在同一总预算下胜过这种重复采样机制; Weibull CDF 在表 1 中也拟合得比对数-S 形更差。个体改进可能带首达色彩, 但宏观最佳-迄今轨迹通过依赖的、有状态的步骤累积, 而非通过无累积进度因子的原始时间危险率。
- **对数-线性:** $\frac{dy}{dt} = b$ (常数)。无饱和项, 它无界增长, 不能在预算内达到许多任务表现出的上限; 它取得了最差的拟合。

因此, 我们解读对数-S 形不仅是“拟合最佳的 S 形”, 而是“其 $y(1 - y)$ 增长律与前沿扩张解读相匹配的曲线”。这是一种机制性偏好, 不是经验性排除——数据无法分离这些对称族。它可通过拐点证伪: 接近 $y = 0.5$ 的对称峰与 logistic(或 probit) 一致; 接近 0.37 的前重峰偏向 Gompertz; 接近 0.63 的后重峰偏向 Weibull。

F 附加相关工作

F.1 不适用于衡量自演化的基准

许多基准评估模型或智能体能否回答问题、完成任务或生成正确产物, 但并不把“自演化”本身作为度量对象。我们以一个狭义的评测意义使用此标签: 这些基准仍可能涉及推理、工具、迭代或环境交互, 但其主要报告量与学习无关。它通常是最终答案准确率、任务成功率、通过率、产物质量、复现保真度或人类时间视野。在这种设定下, 交互通常是产生最终答案或产物的手段, 而非被度量的量。

在最短与最少交互的一端, 许多经典能力基准, 如 MMLU [?], GPQA [?], AIME [?] 以及闭卷

编码或数学评测 [?, ?], 要求模型求解静态问题并报告最终准确率。这些基准可能困难且有用, 但模型的动作并不创造变化的环境, 基准的反馈也不会作为经验暴露给后续的适应。它们因此度量静态知识与推理准确率, 而非自演化。

智能体软件基准通过将智能体置于代码库、开发 workflows 或更大构建任务中提高了真实感, 但许多仍把评测化约为最终结果。SWE-bench [?] 评估现有仓库中的 issue 修复; 发展与演化基准如 RoadmapBench [?] 与 SWE-EVO [?] 评估对现有项目或发布历史的增量变更; 构建或重建基准如 NL2Repo-Bench [?] 与 ProgramBench [?] 评估完整仓库生成或程序行为恢复; SWE-AGI [?] 在固定的 MoonBit API 支架下评估规格驱动的系统构建。FrontierCode [?] 通过评估编码智能体是否在仓库特定质量细则下产生可维护的、可合并的变更, 从而抬高了“生产就绪度”的门槛。它们的任务格式不同, 但共同的评测目标是最终补丁正确性、仓库质量、程序行为、可维护性或测试通过率, 而非智能体通过反馈改进的轨迹。

一些评测在不成为智能体学习基准的前提下拓宽了真实感。GDPval [?] 例如测量模型在跨职业的经济价值专业交付物上的表现。Agents' Last Exam [?] 是真实感与自主性上的近距离对比者: 它在真实 Windows 或 Linux 沙箱中、具备 CLI 与 GUI 访问、专业软件、隐藏参考与可验证最终产物的条件下, 评估通用计算机使用智能体。区分在于评测目标而非智能体性。Agents' Last Exam 面向完成: 智能体为最终交付物工作, 基准报告产物被评分后的最终分数、通过率、成本或时间。其轨迹对回放、审计与失败分析是有价值的, 但“从基准反馈在运行中学习”并非主要被度量的量。

其他基准使用真实或长视野 workflows, 这些 workflows 本可支持学习, 但其已发布协议主要度量最终结果。HCAST [?] 与 METR 时间视野评测 [?] 按人类任务时长校准模型或智能体成功率。TerminalBench [?], RE-Bench [?], PaperBench [?], CORE-Bench [?], PRBench [?], ReplicationBench [?], Collider-Bench [?] 与 ScienceAgentBench [?] 涉及可执行软件任务、终端 workflows 或科学复现。这些设定比短时闭卷任务更兼容学习, 但其中心问题通常是任务是否完成、产物是否工作、论文是否复现, 或最终分数在预算下是否足够高。PaperBench 是一个有用的区分示例: 它使用详细评分细则与许多可评分子任务, 从零开始评估对 20 篇 ICML 论文的智能体复现, 但报告分数仍然是复现质量, 而非每单位反馈的改进。

F.2 适用于衡量学习或自演化的基准

另一条工作线更适合衡量学习或自演化, 因为它暴露了新上下文、序列化流、迭代反馈或长可执行工作区。这些基准是与 EdgeBench 在概念上最近的对比者, 但它们在“什么创造了经验流”与“最终评分什么”上有所不同。我们按评测接口对其进行分组: 从提供的上下文中学习、在序列化流或任务序列上学习、以及迭代优化。这种分组是描述性的, 并不按交互强度排序。序列化基准可以高度交互, 优化基准可暴露诊断反馈; 关键问题是智能体自身的行为是否塑造了它在一个长可执行任务内收到的未来经验。

上下文学习基准 研究最少智能体形式的适应。CL-bench [?] 与 CL-bench Life [?] 测试模型能否使用新提供的专业或个人上下文, 以回答下游问题或执行基于该上下文的任务。这算作从外部信息中学习, 但 episode 流通常并非由智能体自身的动作塑造, 也无变化的环境。模型被要求使用上下文, 而非通过持续交互发现并利用反馈。

序列化学习基准 引入了流式结构。EvaLearn [?] 将 648 个问题分组为 182 个序列, 按顺序评估学习能力与效率。Continual Learning Bench [?] 显式报告序列化经验上的改进, 并引入了与 第 5.2 节类

似的有状态-无状态比较。然而,其合成任务序列具有显式的子任务边界,而 EdgeBench 使用单一连续问题。我们因此按时间而非子任务进行划分,这改变了有状态与无状态设定的定义方式。StreamBench [?] 也报告反馈流上的改进。LLF-Bench [?], LifelongAgentBench [?] 与 Evo-Memory [?] 通过语言反馈、相互依赖的任务、经验回放或演化记忆研究相关问题。这些工作确立了静态能力与学习能力是不同维度。差异并非它们的交互必然更弱;而是序列通常被基准组织为一系列实例、任务、反馈事件或记忆更新。在 EdgeBench 中,序列是单一长任务的**内生序列**:智能体规划、编辑制品、提交尝试、接收诊断,由此影响它接下来学到什么。

迭代优化基准 与 EdgeBench 更为接近,因为它们将重复尝试与经验反馈置于性能的核心。FrontierEng [?] 研究跨生成优化周期的改进,而 MLS-Bench [?] 与 Frontier-CS [?] 评估带可见指标、提交或轨迹级报告的测试时发现或开放式 CS 优化。MLE-bench [?] 自然地属于该家族:智能体在自由的 Kaggle 风格工作区中工作,论文含有关于性能如何随资源扩展的研究,其 100 小时分析对智能体最佳尝试按已用时间打分。ALE-Bench [?] 是另一个由固定目标驱动的邻近长视野设定,即便其原始协议主要报告算法工程上的结果而非学习指标。

最接近的先前工作。 FrontierSWE [?] 与 AutoLab [?] 是最接近的先前工作。两者均评估跨可执行制品、跨重复编辑、实验与经验反馈的**长视野智能体改进**。FrontierSWE 聚焦软件工程与性能调优任务,跨模型与任务类别的平均智能体运行时间约为 3-4 小时。AutoLab 为智能体提供工作的但刻意非最优的程序,跨系统、CUDA、模型开发与谜题式优化任务;它直接度量迭代改进,且不应被视为非学习基准,尽管性能曲线并未作为主要结果报告,且大多数当前任务运行 2-4 小时,仅小部分超过 6 小时。在“长视野智能体任务”的共同前提下,主要区别在于领域覆盖:FrontierSWE 集中于软件工程,AutoLab 强调以优化为中心的研究与工程任务,而 EdgeBench 跨更广泛的可执行领域。EdgeBench 还采用日级任务合约,并将时间对齐的轨迹本身作为主要度量对象,辅以改进面积、回退与主动学习跨度的指标。

综上,这些基准覆盖了问题的重要部分:真实可执行工作、跨组织化流的学习,以及反馈下的迭代优化。EdgeBench 瞄准其交集:在长视野可执行环境中评估 run 内的自演化,经验流由任务自身产生,智能体能影响其接下来观察的内容,且评测使用通用目的智能体沙箱而非基准特定的学习支架。其轨迹级指标报告改进、回退、主动学习跨度与同一连续运行上的最终性能。

F.3 LLM 与智能体的缩放规律

经典神经与语言模型缩放规律主要研究预训练,将“可约损失”建模为模型规模、数据与训练算力的函数。Kaplan 等 [?] 显示语言建模损失在多个尺度轴上遵循幂律关系;Chinchilla 风格的工作 [?] 精化了模型参数与训练 token 之间的算力最优分配。近期工作研究基准性能(而非损失)如何随规模变化;由于准确率类指标是有界的,这些曲线通常由 S 形或其他饱和形式更好地描述 [?, ?, ?, ?]。我们在第 3 节中的对数-S 形环境学习曲线在数学形式上最接近这条“有界性能”路线,但自变量是“任务内已用环境交互”而非预训练算力或模型规模。

测试时与推理时缩放提供了第二个比较点:不同测试时方法展现其自身的经验缩放行为。一条线扩展纯重复采样:HumanEval [?] 引入了 pass@k 作为代码生成的重复采样评测;AlphaCode [?] 使用大规模采样、过滤与选择来提升竞争性编程的解题率;Large Language Monkeys [?] 显示当输出可自动验证时,重复采样可跨多个数量级扩展覆盖率。第二条线研究测试时算力应如何在搜索、修订、投票与验证器/奖励引导策略间分配 [?, ?]。第三条线扩展推理模型中的长链思维推理:OpenAI 的 o1

报告 [?] 将训练时强化学习算力与测试时思维算力作为显式缩放轴;DeepSeek-R1 [?] 研究了强化学习如何引发 LLM 的推理行为。这些研究大多在传统数学、代码、证明或游戏式任务上扩展“产生或选择答案所花费的算力”,报告的函数形式通常是局部对数-线性趋势、指数化幂律或算力分配的前沿。

EdgeBench 反而扩展交互时间:它测量智能体的最佳-迄今性能如何随可执行环境中已用时间的增加而变化,以及智能体如何反复读取、行动、接收反馈并修订。这相比“从静态提示采样或从答案中选择”提供了更广泛的覆盖与不同的缩放对象。

近期智能体测试时缩放工作通过允许智能体随时间从环境中获取信息而更接近我们的设定。OpenAI 的 CUA 报告 [?] 观察到当允许更多步骤时,计算机使用性能提升;BrowseComp [?] 报告了浏览努力的平滑增益。Thinking vs. Doing [?] 明确将交互长度框定为 Web 智能体的测试时缩放轴。在精神上最接近,Mang 等 [?] 在一项长视野编码竞赛设定中比较智能体与人类,在该设定中智能体可随时间尝试、观察与修订。他们报告了有信息量的人类参考曲线,并发现当前智能体进入平台期而人类持续改进。然而,其度量集中于一个竞赛风格的领域与有限任务集。人类改进曲线也难以推断:观测段似乎更接近随时间的线性改进,但可能仅覆盖了更长 S 形轨迹的早期部分。

缩放也已在强化学习中被研究。Hilton 等 [?] 引入了“内在性能”以获得环境交互、模型规模与 RL 性能间的平滑幂律关系。近期 LLM RL 缩放工作对后训练拟合 S 形算力-性能曲线,并使用较小运行预测较大 RL 运行 [?]。RL 与我们的评测均度量从环境反馈(而非人类标注数据)中的学习。实际差异在于分辨率与覆盖。RL 学习运行代价高昂,因此缩放证据通常在更窄的任务分布、更少的不同环境与更少的长轨迹上收集。在 EdgeBench 中,学习算法是上下文学习 (In-Context Learning, ICL):智能体将观察、诊断与先前尝试吸收进其上下文,并用其改进后续动作。由于 ICL 比 RL 训练更便宜地可重复,EdgeBench 可跨 134 个可执行任务环境、多个领域、完整时间对齐轨迹与重复试验来度量环境学习,提供的环境覆盖显著大于 RL 缩放研究中常见的水平。所得拟合超过对 RL 缩放的有界性能类比:它们提供了关于智能体如何从交互中学习的更高分辨率度量。

G 附加基准与实验细节

G.1 “有/无经验”曲线的估计

本节描述第 5.2 节中两条曲线的估计方式。“有经验”曲线对每个任务的三条 12 小时最佳-迄今曲线求平均,然后跨任务求平均。对“无经验”曲线,设 $u_1, \dots, u_n$ 为某任务上 n 次独立尝试的分数;在已用时间 $t = k\tau$ 时,我们将 k 次尝试中最佳者的期望估计为

$$\hat{u}_{k\tau} = \mathbb{E}_S \left[\max_{i \in S} u_i \right], \quad (21)$$

其中 S 从 n 次尝试的大小为 k 的 $\binom{n}{k}$ 子集中均匀采样。这是 pass@k 估计器 [?] 的“分值形式、无放回”扩展;当每个 u_i 为二元时,它退化为通常的 pass@k 估计。

G.2 引力波案例研究细节

所选里程碑压缩了运行到可解读的阶段跃迁,而非每次提交。第一个所选里程碑已经通过了协议与入口检查,在标准网格上生成了全部五个所需的 CSV 文件,并取得 42.8 分。一个后续的关键更新将分数提升至 47.1,主要通过改进源动力学。大约在第 1 小时,另一个里程碑达到约 50,此时智能体改进了 H1/L1 时频谱与 H1 在时域上的早期对齐。这些增益反映了标准信号处理:滤波、归一化、时频平面上的加窗、插值到标准网格以及围绕事件裁剪。

最大的分数增益来自高权重的源动力学组件。 大约在第 4-5 小时, 总分从约 52.3 上升至 59.7, 几乎完全由源动力学子分 (从 64.2 至 89.0) 驱动。由于该子任务权重最大, 一个紧凑的物理模型加上校准产生了大的总分收益。在此之后, 智能体转向 H1 波形调优,H1 时序子分从大约 47 提升至 95, 直到运行结束。

最终方案在 H1 波形与源动力学上很强, 但仍有提升空间以趋向人类标准的 LIGO 风格分析。表 5 拆分了最终分数。GPT-5.5 在 H1 波形重构与源动力学上得分较高, 但在时频谱与 L1 重构上仍较弱。经验校准与参数搜索产生了可观的分数, 但它们并未取代用于应变预处理、白化、时频分析与跨探测器波形对齐的一致端到端流水线。

阶段	时间	最佳分	主要改进
初始里程碑	0 h	42.8-47.1	构建可评测的重建流水线; 替换一个噪声较大的频率估计将推断的源运动从 51.1 降至 42.8
早期信号处理	1-4 h	52.3	围绕合并重定时间--频率窗口; 汉福德探测器波形与两个探测器时频谱共同改进, 将最佳分从 47.1 提升至 52.3
主要突破	4-5 h	59.7	校准双源模型; 轨道速度与分离分数从 64 跳至 89(+25 pp.)
晚期波形调优	5-11.5 h	66.9	发现汉福德探测器波形中的剩余失配; 时间偏移对齐与误差拟合将汉福德从 47.1 提升至 66.9
最终精修	12 h	67.0	对汉福德与利文斯顿探测器的最终修正; 一个小的利文斯顿波形增益将总分从 66.9 提升至 67.0

表 4. 引力波演化轨迹的主要阶段。该轨迹展示了智能体如何在 12 小时运行内精炼信号处理、源动力学与按探测器的波形重构。

组件	最终子分
H1 时序	95.0
L1 时序	57.1
H1 时频谱	42.7
L1 时频谱	44.7
速度/分离	89.0
聚合分	67.0

表 5. GPT-5.5 引力波 12 小时运行的最终子分构成。

G.3 沙箱级续接消融

长运行智能体评测不仅依赖于模型能力, 还依赖于沙箱如何保持运行存活并在多小时内携带有用状态。在 12 小时任务中, 智能体可能因空闲而意外停止并需要恢复。因此, 这些续接机制是实际度量设置的一部分: 薄弱的支架可使有能力的模型停止在任务上工作, 而更强的支架可能帮助同一模型保持活跃并改进。作为补充的沙箱级消融, 我们在固定 12 小时预算下用 GPT-5.5 与 GPT-5.4 评测了两种这样的机制: ‘/goal 模式’ 与 ‘Ralph 循环’。

基线使用标准沙箱: 一个持续的智能体会话、一个防止自愿提前退出的停止钩子, 以及针对异常退出的自动恢复。在 ‘/goal 模式 [?]’ 中, 沙箱提示智能体在开始时创建一个任务级目标、在运行期间保持其活跃, 以及仅在任务被验证时将其标记为完成。**Ralph 循环**遵循 “文件支撑的新鲜上下文” 模式 [?]: 每个循环在同一工作区上启动新的智能体调用, 要求其读取并更新 progress.md, 然后在下一个循环之前将判官反馈附加到该文件。它使用最多 100 个循环、每循环 7200 秒上限以及相同的 12 小时总预算。每个单元格调度三次运行并报告有效运行的平均分。

如表 6 所示,goal 模式与 Ralph 循环常优于基线设置,表明长视野智能体受益于跨扩展运行保留并更新任务状态的沙箱支持。在显示任务平均中,GPT-5.5 从基线的 42.6 提升至 Goal 的 43.1 与 Ralph 的 43.4;GPT-5.4 从基线的 26.1 提升至 Goal 的 31.8 与 Ralph 的 27.6。增益并非在所有任务与模型上均匀,因此我们将此视为附录级沙箱诊断而非主要结果。

任务	GPT-5.5			GPT-5.4		
	Base	Goal	Ralph	Base	Goal	Ralph
portfolio_risk_calibration	25.0	27.5	34.3	10.7	19.8	12.2
storyboard_ad_copywriting	77.0	88.3	97.3	65.7	78.0	83.3
arc_compiler_runtime	72.4	70.6	52.6	50.0	47.2	51.2
battery_soh_rul_anomaly	30.2	23.8	48.8	14.7	14.6	13.4
borden_source_inversion	38.5	32.8	62.2	8.0	17.3	23.2
capecod_plume_reconstruction	16.4	16.0	17.3	12.6	13.3	13.7
combinatorial_games_formalization	38.2	45.3	39.8	17.8	13.0	20.2
ffmpeg_swscale_reimplementation	15.3	16.4	25.2	13.9	28.3	21.8
flt_regular_formalization	75.1	66.7	58.6	48.3	43.7	43.7
git_rewrite_in_zig	18.4	20.5	22.0	15.4	18.4	15.8
tryst_text_adventure	55.7	56.2	40.2	44.3	32.9	42.4
openttd_transport_ai	28.1	39.8	30.6	11.9	1.2	8.1
pocketbase_backend_architecture	62.5	62.5	41.7	20.8	54.2	0.0
symbolic_integration_engine	44.0	36.6	37.7	30.9	63.7	37.2
Avg.	42.6	43.1	43.4	26.1	31.8	27.6

表 6. 沙箱级 “/goal 模式” 与 “Ralph 循环” 的续接消融。GPT-5.5 与 GPT-5.4 在固定 12 小时预算下以 Base/Goal/Ralph 设置进行评测; 每个值为有效运行上的平均分;Avg. 行对显示的任务行求平均。完整单元格在附录 G.3 中详述。**加粗**为最佳设置,下划线为次佳设置。

G.4 逐任务设计备注

本附录给出 134 个 EdgeBench 任务的逐任务设计备注。详见原始 PDF 附录 G.4(对应于原文第 47–51 页)。为节省篇幅,这里汇总各家族的代表性任务与设计意图,完整 134 项的英文版设计备注保留在原始论文中。

- **系统与软件工程** (36 任务): 代表性任务包括 `ann_vector_search_qps`(在硬召回约束下以 NumPy 暴力最近邻基线替换为高性能近似最近邻实现,按 QPS 打分)、`arc_compiler_runtime`(为新编程语言实现完整 TypeScript 编译器流水线)、`ffmpeg_swscale_reimplementation`(在 Rust 中重写 FFmpeg 的 libswscale 像素格式转换与缩放库)、`git_rewrite_in_zig`(以 Zig 二进制复刻 git 行为,产生相同的 CLI 输出、退出码与仓库状态)、`quic_transport_stack`(实现 QUIC 传输协议 RFC9000 子集) 等。
- **科学问题与机器学习** (39 任务): 代表性任务包括 `capecod_plume_reconstruction`(从稀疏监测井重构多分析物地下水羽流)、`cylinder_wake_prediction`(在 CPU 上实现二维圆柱尾流求解器,针对不可压缩 Navier-Stokes 方程)、`gravitational_wave_signal_detection`(复现 LIGO GW150914 引力波检测流水线: 白化、带通滤波、匹配滤波、时频分析与残差计算)、`noisy_product_matching`(判断不同来源的产品列表对是否指代同一真实物品)、`pv_power_forecasting`(基于历史输出与天气特征预测多站点光伏发电) 等。

- **组合优化** (19 任务): 代表性任务包括 `ad_placement_optimization`(将大整数网格划分为不重叠的矩形)、`sat_smt_solver`(从零构建 SAT/SMT 求解器, 实现冲突驱动子句学习、监视文字与重启策略)、`vehicle_routing_time_windows`(为 Solomon 风格基准实现带时间窗的容量受限车辆路径求解器) 等。
- **专业知识工作** (19 任务): 代表性任务包括 `portfolio_risk_calibration`(实现多模块组合管理系统: 风险校准、约束优化、执行成本建模、动态再平衡)、`cross_border_investment_ppt`(生成 15 个月全球跨境投资政策发展的演示文稿)、`storm_claim_ring_audit`(构建灾后保险索赔的欺诈检测流水线)、`equity_objection_report`(生成一份适合法庭提交的法律研究报告) 等。
- **形式化数学与定理证明** (13 任务): 代表性任务包括 `lean_analysis_proofs`(在多文件 Lean4 项目中完成实分析与泛函分析结果的证明义务)、`erdos392_formalization`(在 Lean4 分析数论项目中完成 Erds Problem 392 的证明义务)、`godel_incompleteness_formalization`(在 Lean4 形式化中完成 Gödel 第一不完备性定理的证明义务)、`pfr_formalization`(在 Lean4 形式化中解决多项式 Freiman–Ruzsa 猜想的证明义务) 等。
- **交互式游戏与模拟器** (8 任务): 代表性任务包括 `nethack_dungeon_agent`(通过 NLE 沙箱为 NetHack 实现决策策略)、`openttd_transport_ai`(为 OpenTTD 编写 AI 脚本以构建道路、铁路与航空运输网络)、`wesnoth_tactical_ai`(为 Battle for Wesnoth 编写战术 AI 逻辑) 等。

G.5 逐任务学习曲线

图 14-?? 展示了基准中所有任务的学习曲线, 按能力家族分组。每张图显示原始任务分数与已用时间跨 12 小时预算; 实线为最佳-迄今包络, 淡色虚点为单次提交, 标记最佳后的淡色段表示后期无改进的提交。在少数任务上, y 轴被缩放到主分数带, 极端离群提交位于轴外 (在每张图下方注明)。比较了五个智能体: Claude Opus 4.8、GPT-5.5、GPT-5.4、GLM-5.1 与 DS-V4-Pro。正文中展示的 18 个代表性任务 (图 4) 不在此重复。

G.6 逐任务分数表

下表给出所有任务-模型配置的分。

任务 (系统工程, 节选)	Opus 4.8	GPT-5.5	GPT-5.4	GLM-5.1	DS-V4-Pro
<code>codeflash_repair_performance</code>	57.2±21.9	100.0* ±0.0	—	—	30.5±17.9
<code>ffmpeg_swscale_reimplementation</code>	21.1 ±7.9	15.3±2.8	13.9±3.1	2.2±3.9	3.8±5.9
<code>git_rewrite_in_zig</code>	23.1±2.2	18.4±0.4	15.4±2.4	23.5 ±1.9	17.9±2.1
<code>arc_compiler_runtime</code>	52.0±0.1	72.4 ±15.1	50.0±0.7	48.7±3.5	44.2±4.7
<code>ann_vector_search_qps</code>	59.7 ±2.1	40.7±18.8	50.2±17.4	38.3±17.5	23.8±3.0
<code>rust_multicrate_reconstruction</code>	—	57.8 ±16.8	21.4±3.0	38.5±21.4	23.6±2.4
<code>dependent_type_checker</code>	44.7 ±3.7	24.7±21.4	3.8±6.7	1.9±3.2	0.0±0.0
<code>entt_graph_module</code>	100.0 ±0.0	94.3±3.0	78.4±25.7	100.0 ±0.0	92.0±10.5
<code>stream_processing_engine</code>	100.0 ±0.0	100.0 ±0.0	100.0 ±0.0	100.0 ±0.0	100.0 ±0.0
<code>quic_transport_stack</code>	48.3±14.3	63.6 ±8.9	47.1±16.1	52.5*±22.4	33.4±10.0
<code>zstd_api_modernization</code>	100.0 ±0.0	95.8±7.2	100.0 ±0.0	100.0 ±0.0	91.7±14.4

表 7. 系统工程任务上模型表现。值为最多三次有效运行上的均值; 相邻项在至少两次有效运行时显示 $\pm s$ 。加粗为该任务最佳模型, 下划线 为次佳模型, — 表示无有效结果, * 标记少于三次有效运行。

完整 134 项任务的分数表请见原文表 8–13(对应于原文第 75–78 页), 这里仅展示代表性节选以节约版面。

任务 (科学/ML, 节选)	Opus 4.8	GPT-5.5	GPT-5.4	GLM-5.1	DS-V4-Pro
capecod_plume_reconstruction	19.9±11.1	16.4±5.5	12.6±4.6	10.9±2.5	8.8±0.4
battery_soh_rul_anomaly	37.2±10.7	30.2±14.8	14.7±1.8	18.2±4.3	14.0±0.6
borden_source_inversion	48.4±7.3	38.5±14.3	8.0±1.4	15.1±13.7	38.2±3.6
cylinder_wake_prediction	66.6±4.3	69.9±4.6	39.8±16.6	36.9*±36.0	24.2±14.6
noisy_product_matching_pipeline	68.1±6.5	64.3±6.8	27.1±24.7	44.7±3.4	54.2±9.1
neural_net_weight_recovery	94.9±8.9	100.0*	100.0*±0.0	69.2±0.0	65.4*±5.4
gravitational_wave_signal_detection	61.5±8.1	64.5±2.2	50.0±3.7	44.8±15.7	58.8±2.9
polyimide_homo_lumo_prediction	86.7±23.1	100.0*±0.0	100.0*±0.0	60.0*±0.0	80.0*±28.3
barnes_hut_nbody_acceleration	78.4±6.2	91.6±14.5	87.4±15.4	64.1±8.5	67.1±2.2
csi_time_series_forecasting	83.4±10.4	76.3±2.5	77.3±0.2	44.6±3.1	40.4±12.3

表 8. 科学/ML 任务上模型表现 (节选)。完整表格见原文表 9。

G.7 致谢

我们感谢 UniPat 协助开发 EdgeBench 系统与软件工程部分的 13 个任务; 感谢 Xiaoxing Wu、Xiang Gao、Gao Liu、Yue Yang、Wen Heng、Weinan Zhao、Mailun Gao、Zongbao Zhang 与 Yuchen Wu 在项目期间做出的有益辅助贡献; 感谢 Xinkai Zhou 与 Qi Zhao 在项目期间进行的有意义的讨论; 感谢 Tenglong Ao、Ao Zhang、Shengjie Luo、Zeyi Zhang、Guhao Feng、Tianle Cai、Xinrong Zhang、Yizhong Wang 与 Ruinian Chang 在 EdgeBench 项目之前进行的有意义讨论与合作。

作者贡献

角色	贡献者
核心贡献者	Peng Tang; Deyao Zhu; Chengyao Tang; Xin Zhou; Shanyu Wu; Shengling Qin; Huanyu Zheng; Xuekai Zhu; Yu Liu; Hangliang Ding; Liya Zhu; Shu Zhong [†] ; He Wang; Ming Ding
理论	Zixin Wen
数据基础设施	Ziyu Wan
数据收集与策划	Hao Liu; Zhonglin Xie; Sibao Wang; Chenhui Gou; Haotian Zhu; Linxuan Ren; Xintian Zhang; Yueyang Wang; Nan Chai; Junfeng Zhong; Yipeng Liu; Rui Liu; Panhao Lai; Tian Gao; Yangguang Lin
推荐	Jingyuan Zhang; Sihang Yuan; Maojia Song; Zixin Su; Xuan Qi [‡] ; Ge Zhang; Jinhong Wu [‡] ; Wangchunshu Zhou; Chenyang Zhang [‡] ; Yantao Du; Yinzhu Piao; Ziru Niu
顾问	Hongbin Lin; Wenhao Huang; Lingxiang Meng; Guang Shi

* 等同贡献; [†] 项目负责人; [‡] 外部贡献者。

参考文献

1. B. P. Abbott et al. Observation of gravitational waves from a binary black hole merger. *Physical Review Letters*, 116(6):061102, 2016.
2. Anthropic. The Claude Model Family. Anthropic 技术报告, 2024.
3. Anthropic. Claude Opus 4.8 System Card. 2026 年 5 月发布;2026 年 6 月更新.
4. Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, 等. Continual learning bench: Evaluating frontier AI systems in real-world stateful environments. arXiv:2606.05661, 2026.
5. Per Bak, Chao Tang, and Kurt Wiesenfeld. Self-organized criticality: An explanation of $1/f$ noise. *Physical Review Letters*, 59(4):381–384, 1987.
6. Joseph Berkson. Application of the logistic function to bio-assay. *Journal of the American Statistical Association*, 39(227):357–365, 1944.
7. Akshita Bhagia, Jiacheng Liu, Alexander Wettig, David Heineman, Oyvind Tafjord, Ananya Harsh Jha, Luca Soldaini, Noah A. Smith, Dirk Groeneveld, Pang Wei Koh, Jesse Dodge, and Hannaneh Hajishirzi. Establishing task scaling laws via compute-efficient model ladders. arXiv:2412.04403, 2024.
8. Chester I. Bliss. The method of probits. *Science*, 79(2037):38–39, 1934.
9. Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. arXiv:2407.21787, 2024.
10. Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: Evaluating machine learning agents on machine learning engineering. *ICLR*, 2025.
11. Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.
12. Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, et al. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. *ICLR*, 2025.
13. Ching-An Cheng, Andrey Kolobov, Dipendra Misra, Allen Nie, and Adith Swaminathan. LLF-bench: Benchmark for interactive learning from language feedback. arXiv:2312.06853, 2023.
14. Yizhe Chi, Deyao Hong, Dapeng Jiang, Tianwei Luo, Kaisen Yang, Boshi Zhang, et al. Frontier-Eng: Benchmarking self-evolving agents on real-world engineering tasks with generative optimization. arXiv:2604.12290, 2026.
15. Evan Chu, Rajan Agarwal, Abishek Thangamuthu, Brendan Graham, and Justus Mattern. FrontierSWE: Benchmarking coding agents at the limits of human abilities. *FrontierSWE* 技术报告, 2026.
16. DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948, 2025.
17. DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. 2026 年 4 月技术报告.
18. Jingzhe Ding, Shengda Long, Changxin Pu, et al. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents. arXiv:2512.12730, 2025.

19. Shihan Dou, Ming Zhang, Chenhao Huang, Jiayi Chen, Feng Chen, Shichun Liu, Yan Liu, Chenxiao Liu, Cheng Zhong, Zongzhang Zhang, et al. EvaLearn: Quantifying the learning capability and efficiency of LLMs via sequential problem solving. *arXiv:2506.02672*, 2025.
20. Shihan Dou, Yujiong Shen, Chenhao Huang, Junjie Ye, Jiayi Chen, Junzhe Wang, Qianyu He, Shichun Liu, Changze Lv, Jiahang Lin, et al. CL-bench Life: Can language models learn from real-life context? *arXiv:2604.27043*, 2026.
21. Shihan Dou, Ming Zhang, Zhangyue Yin, Chenhao Huang, Yujiong Shen, Junzhe Wang, Jiayi Chen, Yuchen Ni, Junjie Ye, Cheng Zhang, et al. CL-bench: A benchmark for context learning. *arXiv:2602.03587*, 2026.
22. Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv:2407.21783*, 2024.
23. Darius A. Faroughy, Sofia Palacios Schweitzer, Ian Pang, Siddharth Mishra-Sharma, and David Shih. Collider-Bench: Benchmarking AI agents with particle physics analysis reproduction. *arXiv:2605.13950*, 2026.
24. Gemini Team. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv:2403.05530*, 2024.
25. Gemini Team. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv:2507.06261*, 2025.
26. GLM-5 Team. GLM-5: from vibe coding to agentic engineering. *arXiv:2602.15763*, 2026.
27. Benjamin Gompertz. On the nature of the function expressive of the law of human mortality, and on a new mode of determining the value of life contingencies. *Philosophical Transactions of the Royal Society of London*, 115:513–583, 1825.
28. Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *ICLR*, 2021.
29. Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*, 2021.
30. Jacob Hilton, Jie Tang, and John Schulman. Scaling laws for single-agent reinforcement learning. *arXiv:2301.13442*, 2023.
31. Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv:2203.15556*, 2022.
32. Geoffrey Huntley. Everything Is a Ralph Loop. 博客文章, 2026 年 1 月 17 日.
33. Yuki Imajuku, Kohki Horie, Yoichi Iwata, Kensho Aoki, Naohiro Takahashi, and Takuya Akiba. ALE-bench: A benchmark for long-horizon objective-driven algorithm engineering. *arXiv:2506.09050*, 2025.
34. Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *ICLR*, 2024.
35. Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.
36. Devvrit Khatr, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S. Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning

compute for LLMs. arXiv:2510.13786, 2025.

37. Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al. Measuring AI ability to complete long tasks. arXiv:2503.14499, 2025.
38. Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. arXiv:2203.07814, 2022.
39. Eric Lu, Ben Pan, Deniz Birlikci, Sam Lee, Ray Wang, Rohan Choudhury, Fermi Ma, TC Qin, Carlo Baronio, Silas Alberti, et al. Introducing FrontierCode. *Cognition AI 博客*, 2026.
40. Bohan Lyu, Yucheng Yang, Siqiao Huang, Jiaru Zhang, Qixin Xu, Xinghan Li, Xinyang Han, Yicheng Zhang, Huaqing Zhang, Runhan Huang, et al. MLS-Bench: A holistic and rigorous assessment of AI systems on building better AI. arXiv:2605.08678, 2026.
41. Qiuyang Mang, Wenhao Chai, Zhifei Li, Huanzhi Mao, Shang Zhou, Alexander Du, Hanchen Li, Shu Liu, Edwin Chen, Yichuan Wang, et al. FrontierCS: Evolving challenges for evolving intelligence. arXiv:2512.15699, 2025.
42. Qiuyang Mang, Kaiyuan Liu, Bo Peng, Shreyas Pimpalgaonkar, Alex Dimakis, and Alvin Cheung. Humans still beat AI in the long horizon: Revisiting test-time scaling in the agent era. *博客文章*, 2026.
43. Mathematical Association of America. MAA invitational competitions: American invitational mathematics examination (AIME). 2026.
44. Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *ICLR*, 2026.
45. Mark E. J. Newman. Power laws, pareto distributions and zipf’s law. *Contemporary Physics*, 46(5):323–351, 2005.
46. OpenAI. GPT-4 technical report. arXiv:2303.08774, 2023.
47. OpenAI. Learning to reason with LLMs. 2024.
48. OpenAI. Introducing SWE-bench verified. 2024.
49. OpenAI. Computer-using agent. 2025.
50. OpenAI. GPT-4.5 System Card. 2025.
51. OpenAI. Update to GPT-5 System Card: GPT-5.2. 2025.
52. OpenAI. GPT-5 System Card. arXiv:2601.03267, 2025.
53. OpenAI. Follow a Goal. Codex documentation, 2026.
54. OpenAI. GPT-5.4 Thinking System Card. 2026 年 3 月.
55. OpenAI. GPT-5.5 System Card. 2026 年 4 月.
56. David Owen. How predictable is language model benchmark performance? arXiv:2401.04757, 2024.
57. Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simon Posada Fishman, Marwan Aljubeh, Phoebe Thacker, Laurance Fauconnet, et al. GDPval: Evaluating AI model performance on real-world economically valuable tasks. arXiv:2510.04374, 2025.

58. Shi Qiu, Junyi Deng, Yiwei Deng, Haoran Dong, Jieyu Fu, Mao Li, Zeyu Li, Zhaolong Zhang, Huiwen Zheng, Leidong Bao, et al. PRBench: End-to-end paper reproduction in physics research. arXiv:2603.27646, 2026.
59. David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. arXiv:2311.12022, 2023.
60. David Rein, Joel Becker, Amy Deng, Seraphina Nix, Chris Canal, Daniel O’Connel, Pip Arnott, Ryan Bloom, Thomas Broadley, et al. HCAST: Human-calibrated autonomy software tasks. arXiv:2503.17354, 2025.
61. Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. arXiv:2405.10938, 2024.
62. Junhong Shen, Hao Bai, Lunjun Zhang, Yifei Zhou, Amrith Setlur, Shengbang Tong, Diego Caples, Nan Jiang, Tong Zhang, Ameet Talwalkar, and Aviral Kumar. Thinking vs. doing: Agents that reason by scaling test-time interaction. arXiv:2506.07976, 2025.
63. Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. CORE-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. arXiv:2409.11363, 2024.
64. Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. arXiv:2408.03314, 2024.
65. Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI’s ability to replicate AI research. *ICML*, 2025.
66. Yiyu Sun, Xinyang Han, Weichen Zhang, Yuanbo Pang, Tianyu Wang, et al. Agents’ last exam. arXiv:2606.05405, 2026.
67. Minh V. T. Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. arXiv:2512.18470, 2025.
68. Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents. arXiv:2504.12516, 2025.
69. Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, et al. Evo-Memory: Benchmarking LLM agent test-time learning with self-evolving memory. arXiv:2511.20857, 2025.
70. Waloddi Weibull. A statistical distribution function of wide applicability. *Journal of Applied Mechanics*, 18(3):293–297, 1951.
71. Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, et al. RE-bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. *ICML*, 2025.
72. Wikipedia contributors. Sigmoid function. Wikipedia, 2025.
73. Cheng-Kuang Wu, Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung-yi Lee. StreamBench: Towards benchmarking continuous improvement of language agents. *NeurIPS*, 2024.
74. Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem solving with language models. arXiv:2408.00724, 2024.

2024.

- 75. Xinbo Xu, Ruihan Yang, Haiyang Shen, et al. RoadmapBench: Evaluating long-horizon agentic software development across version upgrades. arXiv:2605.15846, 2026.
- 76. Zhangchen Xu, Junda Chen, Yue Huang, Dongfu Jiang, Jiefeng Chen, Hang Hua, Zijian Wu, Zheyuan Liu, Zexue He, Lichi Li, et al. AutoLab: Can frontier models solve long-horizon auto research and engineering tasks? arXiv:2606.05080, 2026.
- 77. John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. ProgramBench: Can language models rebuild programs from scratch? arXiv:2605.03546, 2026.
- 78. Christine Ye, Sihan Yuan, Suchetha Cooray, Steven Dillmann, Ian L. V. Roque, Dalya Baron, Philipp Frank, Sergio Martin-Alvarez, Nolan Koblishke, Frank J. Qu, et al. ReplicationBench: Can AI agents replicate astrophysics research papers? arXiv:2510.24591, 2025.
- 79. Z.ai. GLM-5.1: Towards long-horizon tasks. 2026.
- 80. Hanlin Zhang, Jikai Jin, Vasilis Syrgkanis, and Sham Kakade. Prescriptive scaling reveals the evolution of language model capabilities. arXiv:2602.15327, 2026.
- 81. Zhirui Zhang, Hongbo Zhang, Haoxiang Fei, et al. SWE-AGI: Benchmarking specification-driven software construction with MoonBit in the era of autonomous agents. arXiv:2602.09447, 2026.
- 82. Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. LifelongAgentBench: Evaluating LLM agents as lifelong learners. arXiv:2505.11942, 2025.

EdgeBench: Unveiling Scaling Laws of Learning from Real-World Environments

ByteDance Seed

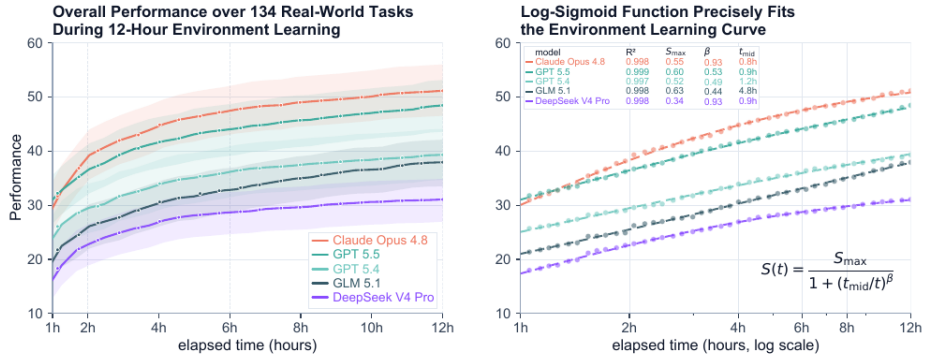


Figure 1 Overall performance follows a precise log-sigmoid relationship with environment interaction time when averaged over all 134 tasks (mean $R^2 = 0.998$).

Abstract

Pretraining scaling laws reveal that model capability improves predictably with data and compute. But learning from real world environments after deployment remains far less understood. Analyzing roughly 38,000 hours of agent interaction with the environment across 134 real world tasks, we find, to the best of our knowledge, the first evidence that overall performance during environment learning follows a log-sigmoid scaling law with remarkably high precision, reaching $R^2 = 0.998$. Across model generations, we also find that agent learning speed roughly doubles every three months. This discovery stems from EdgeBench, a suite of 134 real world tasks with ultra-long horizons, spanning scientific discovery, software engineering, combinatorial optimization, professional knowledge work, formal mathematics, and interactive games. Each task sustains at least 12 hours of continuous agent operation under rich, multilevel feedback, and is built through substantial expert effort. We publicly release 51 tasks and our full evaluation framework to accelerate the study of how agents learn from real world experience.

Date: July 7, 2026

Correspondence: Shu Zhong at zhongshu@bytedance.com

Project Page: <https://edge-bench.org>

图 1. 整体性能随交互时间呈现精确的对数-S 形关系 (134 个真实世界任务求均值, $R^2 = 0.998$)。

EdgeBench: 134 real-world, day-long tasks across 6 capability families

Agent runtime ≥ 12 h per task · recorded human expert effort mean **57.2 h** · every task provides **rich, real-world feedback** for continuously learning from environment

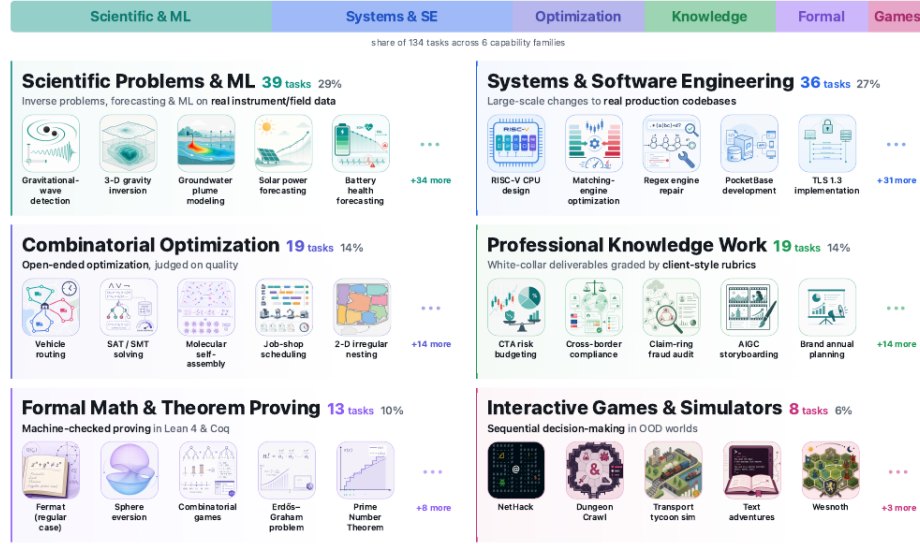


Figure 2 EdgeBench task taxonomy. 134 real world tasks across six capability families, with feedback channels designed to support within-run improvement. Recorded human expert effort estimates: mean 57.2 h.

1 Introduction

Scaling laws for pretraining [31, 35] revealed that model capability improves predictably with data and compute, an insight that guided much of the progress in the years that followed [2, 3, 22, 24, 25, 46, 50–52, 54, 55]. Now, as large language models enter the agent era, they are being deployed into a growing range of real world environments where they can try to learn from interaction, yet whether learning from such environments obeys a clean scaling law remains unknown. Analyzing roughly 38,000 hours of agent environment interaction across 134 diverse real world tasks, we find, to the best of our knowledge, the first evidence that **when agents learn from real world environments, overall performance follows a log-sigmoid scaling law as a function of environment interaction time**, achieving remarkably high precision with $R^2 = 0.998$.

Why study agents’ ability to learn from their environments? Real world use of AI depends on more than what a model learned during training. Some needed knowledge never appears in training data, such as private records and internal tools. Even when raw data exists, it omits the human process behind it: the trial-and-error, the interpretation of evidence, and the adaptation to feedback through which experts actually reach results. The real world also never stands still: human knowledge keeps advancing, and new tools, discoveries, and problems continually emerge that no fixed training corpus can anticipate. Therefore, an agent’s ability to learn from its environment and improve task performance is central to deploying AI systems at scale in the real world.

Studying this ability requires task environments that resemble real use, provide informative feedback for learning, and allow agents enough time to learn through interaction. However, existing benchmarks often lack such feedback or limit agents to only minutes or a few hours, making them not well suited to this goal [10, 15, 34, 44, 66, 76]. To address this gap, we created **EdgeBench**. Our benchmark contains **134 realistic and diverse tasks spanning six capability families**, from scientific research and software engineering to

图 2. EdgeBench 任务分类。134 个真实任务横跨六个能力家族，每个家族均设计了支持“run 内改进”的反馈通道。记录的人工专家投入均值 57.2 小时/任务。

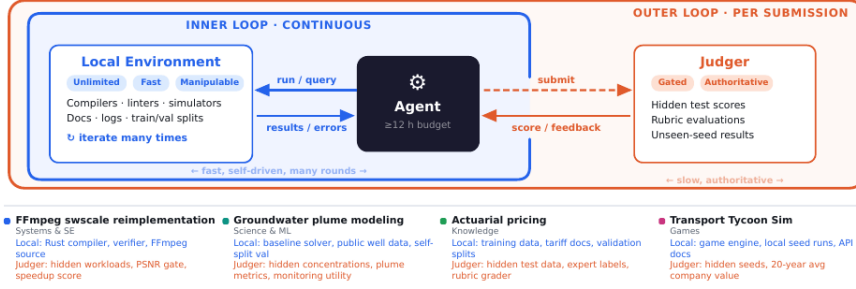


Figure 3 The informative feedback loop in EdgeBench. The inner loop (blue) lets agents iterate freely with local feedback; the outer loop (orange) gates authoritative judge feedback behind submissions. Bottom: representative tasks showing how both feedback channels are instantiated across capability families.

- **Scientific Problems & ML** (39 tasks). Each task uses real world research data and experimental settings sourced from working scientists. Domain expertise is essential: agents must formulate hypotheses, choose models, validate against noisy observations, and refine iteratively. Many problems are open-ended, with no known optimal solution.
- **Systems & Software Engineering** (36 tasks). Agents work on production-grade codebases where a single task may require thousands of lines of change, with over 100,000 lines in the largest cases. Because the code spans interdependent modules, an agent must reason about cross-module coupling while meeting both correctness and performance targets.
- **Combinatorial Optimization** (19 tasks). These are open-ended, predominantly NP-hard problems where exact methods are intractable and progress depends on designing, tuning, and iterating on heuristic search strategies. Even strong solvers have room to improve with additional time and feedback.
- **Professional Knowledge Work** (19 tasks). These tasks reproduce real white-collar deliverables across finance, education, healthcare, and legal domains, matching work that would take a human professional with three or more years of experience roughly three full days to complete. Many tasks feature carefully designed rubrics and multi-round delivery feedback that approximate real client review cycles, so agents can learn from structured critique and revise iteratively.
- **Formal Math & Theorem Proving** (13 tasks). These tasks sit at the frontier of mathematical difficulty and require building large-scale machine-checked proofs in Lean, coupling deep mathematical insight with substantial formal-verification engineering. Most are newly created for EdgeBench and designed to support iterative progress: agents receive structured intermediate guidance and can extend partial proofs incrementally.
- **Interactive Games & Simulators** (8 tasks). These are real games designed for human players, where proficient humans typically invest tens of hours to master the mechanics. The state spaces are enormous and each run is procedurally distinct, so agents face strong out-of-distribution pressure. Agents must develop and refine strategies through high-frequency interaction across many episodes.

Tasks whose primary difficulty lies in visual understanding, especially GUI operation, are excluded. When success depends on the vision backbone rather than iterative reasoning, learning ability and perceptual capability are hard to separate.

图 3. EdgeBench 的信息反馈循环。内循环 (蓝色) 允许智能体基于本地反馈自由迭代; 外循环 (橙色) 将权威判官反馈置于提交门控之后。下方展示了六大家族中该反馈模式的具体实例化。

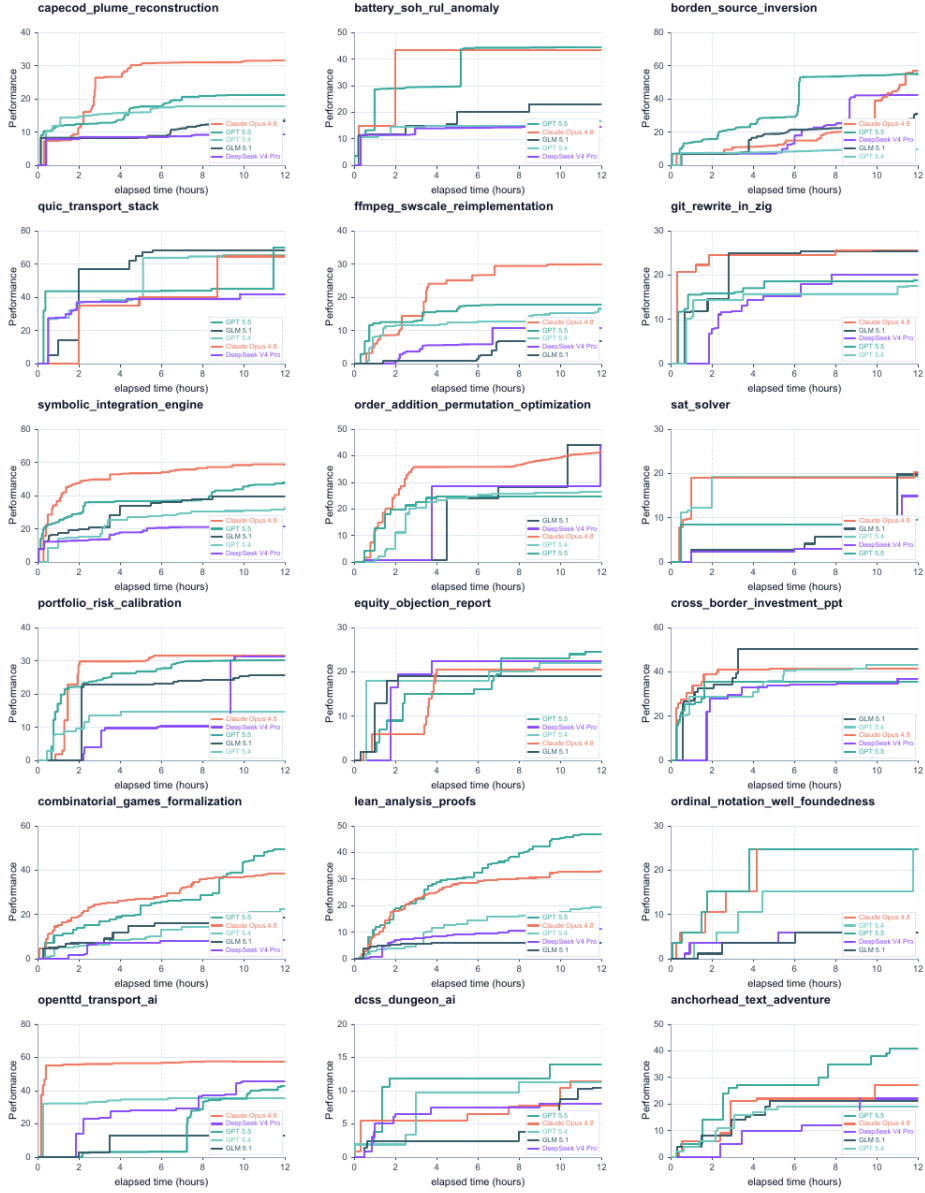


Figure 4 Learning curves over 12 hours for 18 representative tasks across six capability families. The remaining task curves are provided in Figures 15–35.

图 4. 18 个代表性任务跨 12 小时的学习曲线, 涵盖六个能力家族。其余任务曲线见附录图 G.5。

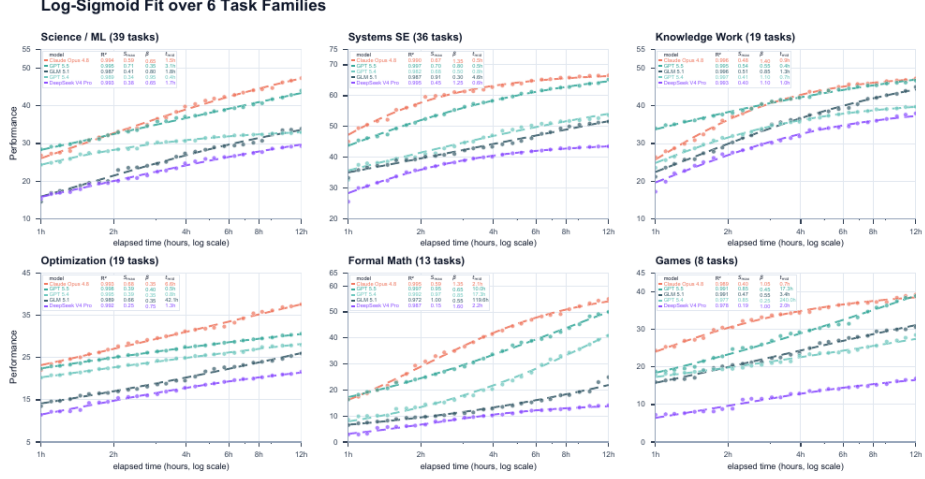


Figure 5 Task-family-level log-sigmoid fits across the six EdgeBench task families. Despite large differences in task type and scoring function, the same log-time sigmoidal form fits the average trajectory in each task family well.¹

Per-task trajectory diversity. Figure 4 shows per-task learning curves for 18 representative tasks, illustrating how different models improve on the same task over time. The full set of per-task curves is provided in Appendix G.5 for all 134 tasks. The tasks span diverse domains and exhibit heterogeneous learning dynamics, with trajectories ranging from smooth incremental gains to long plateaus, abrupt breakthroughs, and irregular regressions.

Aggregate curves reveal a common structure. Although these individual curves are heterogeneous across both tasks and models, their cross-task averages are unexpectedly smooth and share a common structure. Motivated by prior log-sigmoid fits of benchmark performance under pretraining scale-up, we fit the averaged environment learning curves with the following three-parameter log-sigmoid model:

$$S(t) = \frac{S_{\max}}{1 + (t_{\text{mid}}/t)^{\beta}}, \quad (1)$$

where t is elapsed interaction time and $S(t)$ is best-so-far performance. The fitted parameters serve as empirical descriptors of the aggregate learning curve: $S_{\max}$ is the attainable score ceiling, t_{mid} is the interaction time at which the curve reaches half of that ceiling, and β controls how sharply progress concentrates in log time. Thus a smaller t_{mid} means the model reaches the bulk of its attainable score sooner, while a larger β corresponds to a steeper learning transition. Section 3.2 shows that this simple functional form fits the empirical learning curves surprisingly well.

3.2 Log-Sigmoid Curves Fit Environment Learning with Remarkable Precision

We find that the log-sigmoid fit is precise and robust across every setting we tested:

- **Log-sigmoid curves precisely fit the 134-task average for all five models.** As shown in Figure 1, after averaging over all 134 tasks, the fitted log-sigmoid curve closely tracks each model’s 12-hour learning trajectory. The fit is uniformly tight across all five models, with $R^2 \geq 0.997$ in every case.

¹Fit precision improves as the number of fitted tasks increases; see Figure 8.

²GPT-5.4 service availability dropped in the latter half of the experiment, which may partly explain its forecast deviation; see Appendix B.

图 5. EdgeBench 六个任务家族的逐家族对数-S 形拟合。尽管任务类型与评分函数差异巨大，同一对数时间下的 S 形形式仍能良好拟合每个家族的均值轨迹。

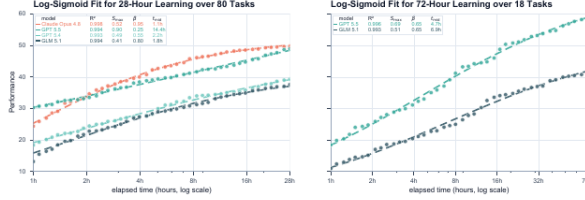


Figure 6 Long-horizon log-sigmoid fits beyond the main 12-hour benchmark window. The left panel fits 28-hour trajectories averaged over 80 tasks; the right panel fits 72-hour trajectories averaged over 18 tasks. All fitted curves remain highly precise, with $R^2 \geq 0.993$.

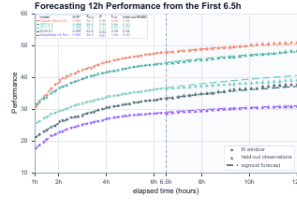


Figure 7 Forecasting 12-hour performance from the first 6.5 hours. Log-sigmoid fits on this early window accurately predict the held-out remainder for all five models.²

- **The same log-sigmoid form persists across heterogeneous task families.** Figure 5 repeats the fit separately across the six capability families in EdgeBench. These families require different knowledge, exercise different capabilities, and produce visibly different aggregate learning curves. Yet each family is still well described by the same log-sigmoid form, including smaller families where fewer tasks make the averaged trajectories noisier.
- **The fit remains precise under substantially longer interaction horizons.** Figure 6 extends the analysis beyond the main 12-hour window to 28-hour and 72-hour interaction horizons. Due to resource limits, the 28-hour fit covers 80 tasks and four models, while the 72-hour fit covers 18 tasks and two models. The same log-sigmoid structure remains stable across both longer settings, with every fitted curve reaching $R^2 \geq 0.993$.
- **The log-sigmoid law exhibits predictive power.** Figure 7 tests this by fitting each 12-hour aggregate curve using only the first 6.5 hours, then evaluates the forecast on held-out observations from 6.5 to 12 hours. The extrapolated curves remain close to the later observed trajectories across all five models, with $R^2 \geq 0.997$ and RMSE below 1.0 performance point in every case.

These precise fits raise two questions: is the log-sigmoid genuinely the right curve, or would any S-shape do, and where does so clean a law come from?

Log-sigmoid fits best among common S-curves. Many saturating growth processes are S-shaped [72]. We therefore compare the log-sigmoid against other common S-curves, each viewing an S-shaped learning curve as a cumulative distribution over a time coordinate: log-probit [8], log-Gompertz [27], and a Weibull CDF [70] on raw time, together with a two-parameter log-linear baseline; for the log-time families the independent variable is $x = \ln t$. We fit every family on the 12h, 28h, and 72h full windows and pool the error. As Table 1 shows, the log-sigmoid family attains the lowest RMSE, while the log-linear baseline is substantially worse. The empirical signal is thus robustly sigmoidal rather than tied to a single link function, and is not explained by mere linear improvement in $\ln t$. Appendix E discusses why we nevertheless prefer the log-sigmoid on mechanistic grounds.

The law emerges from a population of tasks. A single task’s trajectory is noisy and idiosyncratic, yet Figure 8 shows that the log-sigmoid fit becomes steadily more precise as we average over more tasks: the residual error falls monotonically as tasks accumulate, from 1 task to all 134. The clean scaling law is therefore an *emergent, population-level* regularity, sharp only across many diverse tasks rather than within any one of them.

图 6. 用前 6.5 小时预测 12 小时性能以及超长视野拟合。左图对 80 个任务的 28 小时轨迹取平均拟合；右图对 18 个任务的 72 小时轨迹取平均拟合。所有拟合曲线仍非常精确, $R^2 \geq 0.993$ 。

Family	Functional form	RMSE
Log-Sigmoid	$S(t) = \frac{S_{\max}}{1 + (t_{\text{mid}}/t)^\beta}$	0.390
Log-Probit	$S(t) = S_{\max} \Phi\left(\frac{\ln t - \mu}{\sigma}\right)$	0.398
Log-Gompertz	$S(t) = S_{\max} \exp[-\exp\{-c(\ln t - x_0)\}]$	0.402
Weibull CDF	$S(t) = S_{\max} (1 - \exp\{-(t/\lambda)^\beta\})$	0.404
Log-Linear	$S(t) = a + b \ln t$	0.717

Table 1 Full-window fit error for three-parameter S-curve families and a two-parameter log-linear baseline. RMSE is measured in performance points on the 0–100 score scale; lower is better.

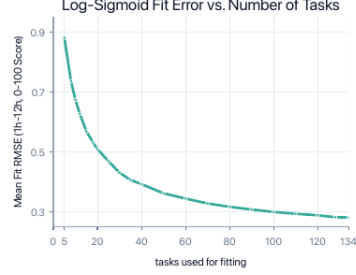


Figure 8 Log-sigmoid fit error decreases as more tasks are averaged.

3.3 A Theory of the Log-Sigmoid Law

The empirical fits show a robust log-sigmoid shape, but do not by themselves explain why this form should appear. We propose a theoretical model: environment learning is a frontier expansion process on the underlying task graphs. In this view, each task is modeled by a latent graph of score units, the already-unlocked units exert influence on the locked neighbors to unlock them, and progress occurs when the frontier between unlocked and locked score nodes advances. Appendix D gives the full derivation; here we summarize the mechanism. Throughout the section, we use

$$u = \log t - \log t_{\text{mid}}$$

as a change of coordinate of the time axis.

Environment learning is a frontier expansion process. For a single task, let us consider the task score is composed of many score units, representing nodes i on the task graph G with score w_i and normalized score weights $\mu_i = w_i / \sum_i w_i$. Let $n_i(u) \in \{0, 1\}$ indicate whether unit i has been unlocked at time u , the normalized score obtained is

$$x(u) = \sum_i \mu_i n_i(u).$$

On the task graph G , an edge weight $K_{ij} \geq 0$ measures how much an unlocked source unit j helps unlock a target unit i . Thus a locked unit i receives an influence field

$$h_i(u) = \sum_j K_{ij} n_j(u).$$

If locked units unlock randomly at an expected rate proportional to this field, then conditioned on the current state,

$$\frac{d}{du} \mathbb{E}[x(u) \mid n(u)] = \eta \sum_{i \in L(u)} \sum_{j \in U(u)} \mu_i K_{ij}. \quad (2)$$

The expected score-growth rate is therefore exactly the weighted frontier cut from unlocked units $U(u)$ to locked units $L(u)$.

Frontier process moves at a speed proportional to $x(1-x)$. The exact frontier cut still depends on the task graph structure. The mean-field approximation is to assume that, at the aggregate level, every macroscopic unlocked-locked cut has approximately product-measure influence:

$$\sum_{i \in L} \sum_{j \in U} \mu_i K_{ij} \approx \kappa \mu(L) \mu(U).$$

图 7. 对数-S 形拟合误差随求平均任务数的增加而降低。

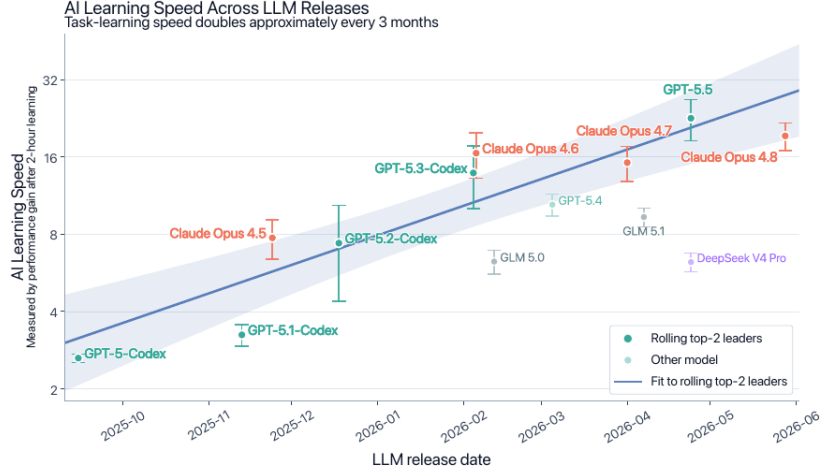


Figure 9 Learning speed across evaluated LLM releases. Learning speed denotes the two-hour performance gain on the fixed 18-task slice. The blue line fits the rolling top-2 leaders by release date and indicates an approximately three-month doubling trend.

frontier speeds, or non-scale-free graph structures. These failure modes clarify the scope of the theory: the log-sigmoid is most natural when progress behaves like a sufficiently mixed frontier expansion process on an approximately fractal structured graph. In this sense, learning from environments tests whether an agent can convert diverse feedback into reusable structure that accelerates subsequent discovery. We view this as central to why environment learning is worth measuring and scaling.

4 Agent Learning Speed Doubles Approximately Every Three Months

Frontier agents can now improve through interaction with task environments. This raises a separate question: are newer models learning from their environments faster? We measure this by comparing how much performance each agent gains over a fixed interaction budget across model release dates.

4.1 Experimental Design

Disentangling prior knowledge from environment learning. A high score may reflect what the model already knew rather than what it learned during the run. To reduce this confound, we select an 18-task slice from EdgeBench where models show similar first-attempt performance. With comparable starting points, later gains provide a cleaner measure of environment learning. We measure task-learning speed as the **average performance gain** over a fixed two-hour budget.

Evaluation protocol. We evaluate frontier open- and closed-source AI systems released from September 2025 through the current evaluation window, when frontier systems became capable of sustained autonomous runs. Each model is run three times per task. GPT models are run with Codex; all other models are run with Claude Code. As shown in Figure 10 (left), models start from comparable first-attempt performance on this 18-task slice, with an average of 6.87 ± 0.97 .

图 8. 跨 LLM 发布日 AI 学习速度的演进。学习速度定义为固定 18 任务子集上的两小时性能增益。蓝色线对发布日的滚动前两名进行拟合，显示大约三个月翻倍的趋势。



Figure 10 Learning outcomes and agent effort on the 18-task slice. Left: normalized initial performance and best performance after two hours. Middle: fraction of submissions that improve the best-so-far score. Right: average number of submissions per task. Error bars in the left panel show uncertainty over three replicate runs.

Model	Overall Score $\uparrow$						Category Score@12h $\uparrow$					
	@2h	@4h	@6h	@8h	@10h	@12h	Science	Code	Optimize	Knowledge	Math	Games
Opus 4.8	39.0	45.7	48.1	49.8	50.9	51.3	48.5	67.4	36.5	47.0	55.0	39.3
GPT-5.5	36.8	<u>42.1</u>	<u>44.5</u>	<u>46.3</u>	<u>47.6</u>	<u>48.4</u>	44.3	<u>65.0</u>	33.6	45.7	<u>50.0</u>	<u>39.1</u>
GPT-5.4	29.7	34.0	36.5	38.0	38.9	39.3	33.5	54.1	27.9	38.8	40.8	29.0
GLM-5.1	26.0	30.4	32.9	34.9	36.5	37.4	33.8	50.9	26.4	43.5	24.6	29.3
DS-V4-Pro	23.3	27.1	29.0	29.9	30.9	31.0	30.0	43.0	21.5	37.0	14.1	16.9

Table 2 Aggregate leaderboard. Overall scores are reported at each time budget, while category scores are reported at the 12-hour budget. Bold marks the best value in each score column and underlining marks the second-best value.

4.2 Learning Speed across Model Generations

Figure 9 reports the two-hour evaluation results on the fixed 18-task slice. The y-axis measures agent learning speed, defined as performance gain over two hours, on a log scale. To estimate the frontier trend, we use darker markers to highlight the top two models at each release date. We then fit a linear trend to these frontier points, with the fitted 95% confidence interval shown as the shaded band.

Figure 9 shows a rapid increase in learning speed across recent model generations. From GPT-5-Codex in September 2025 to GPT-5.5 in April 2026, learning speed increases by roughly $8\times$ over 221 days. A log-linear fit to the frontier models captures this trend well, corresponding to **an approximate doubling every three months**. Figure 10 shows that this improvement is not simply explained by more frequent submissions. Submission frequency (right panel) changes unevenly: newer GPT models submit more actively, while other families do not. The middle panel tells a different story: later models turn a larger fraction of submissions into best-so-far improvements. This trend therefore reflects more effective learning from each interaction, not merely more attempts.

5 Analysis of Environment Learning Dynamics

We study how frontier models perform and learn from environmental feedback over long horizons. We evaluate five frontier models: Claude Opus 4.8, GPT-5.5, GPT-5.4, GLM-5.1, and DeepSeek-V4-Pro (DS-V4-Pro). This section contains four analyses. Section 5.1 compares frontier models at the aggregate, family, task, and submission levels. Section 5.2 tests whether accumulated experience adds value beyond independent restarts. Section 5.3 examines whether longer context still improves long-horizon interaction. Section 5.4 traces a single scientific task to show how an agent’s improvements unfold. Appendix G.3 reports additional harness-level continuation ablations for /goal mode and the Ralph loop.

图 9. 18 任务子集上的学习结果与智能体投入。左: 归一化初始性能与两小时学习后的最佳性能。中: 改进“迄今最佳”分数的提交比例。右: 每个任务的平均提交次数。左图误差棒显示三次重复运行的不确定性。

5.1 Comparison across Frontier Models

Setup. We follow Section 3.1 and analyze 12-hour trajectories from two angles: **(1)** aggregate, family, and task performance, and **(2)** submission efficiency, i.e., how often submissions improve the current best result.

Performance Comparison. Table 2 gives the aggregate leaderboard over the 2 to 12 hour budget and reports family-level means at 12 hours. Claude Opus 4.8 leads throughout the time budget and reaches 51.3 at 12 hours, followed by GPT-5.5 at 48.4. GPT-5.4 and GLM-5.1 form the next tier at 39.3 and 37.4, while DS-V4-Pro obtains 31.0. Family-level results are broadly consistent with the aggregate ranking: Claude Opus 4.8 leads each family mean, with GPT-5.5 especially close in Games and second overall. Detailed per-task performance is reported in Appendix G.6, with * marking cells based on fewer than three valid runs.

Submission efficiency. We count each agent submission and mark it as effective when it sets a new best-so-far score. Figure 11 compares each model’s 12-hour effective-submission rate with its final performance. Models that make more effective submissions usually perform better, but more submissions do not automatically lead to a better final result. **Claude Opus 4.8 achieves the best final performance despite submitting less often than GPT-5.5**, and GPT-5.4 has the highest effective-submission rate but still trails the top two models. Progress therefore depends not only on how often an agent improves its score, but also on whether those improvements are large, reliable, and reusable. Stronger agents use feedback more deliberately: they build a submit-ready baseline, preserve the current best solution, make focused changes, and use feedback to keep gains or roll back failures. Weaker agents more often over-trust local proxies, bundle unrelated edits, or continue broad exploration after feedback has ruled out a direction, reducing sample efficiency.

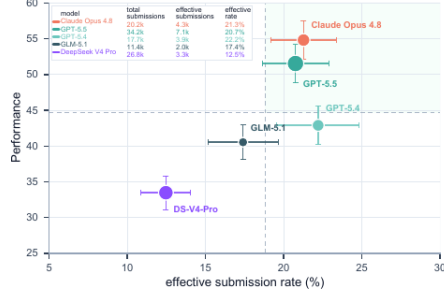


Figure 11 12-hour effective-submission rate versus final performance. Marker area shows total submissions; error bars show task-level standard errors, and shading marks above-average rate and performance.

5.2 Agents Do Learn from Experience beyond Repeated Sampling

Motivation. A rising best-so-far curve does not by itself prove that an agent is learning. Running longer also gives more chances to stumble on a good solution: with enough independent tries, the best of them climbs by luck alone. We therefore test whether the agent’s accumulated experience adds value beyond such repeated sampling under the same total time budget.

Setup. We give Opus 4.8 the same 12-hour budget on each of 17 tasks and compare two ways of spending it, *with* versus *without* accumulated experience. **With experience:** the agent runs once and continuously, keeping its workspace, artifacts, and feedback history throughout, so experience builds up across the whole run. **Without experience:** the same budget is split into $n = 6$ independent attempts of $\tau = 2$ hours, with all state discarded between attempts and only the best result kept, so each attempt starts from scratch and any gain can come only from repeated sampling. Comparing the two at elapsed time $t = k\tau$ contrasts one continuous run after t hours against the best of k independent attempts using the same total time. Appendix G.1 details how each curve is estimated.

Result. Figure 12a shows a clear gain from experience: the with-experience curve stays above the without-experience baseline under the same time budget. At 12 hours it reaches 43.0 versus 36.1, a gain of +6.9. The improvement is therefore not explained by repeated sampling alone: accumulating and reusing task experience drives progress beyond what independent restarts achieve.

图 10. 12 小时有效提交率与最终表现的对比。标记面积表示总提交数; 误差棒表示任务级标准误; 阴影标记高于均值水平。

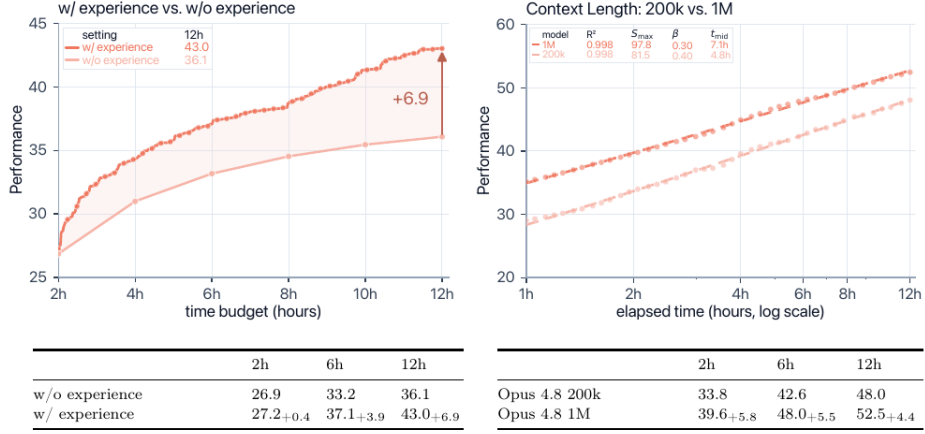


Figure 12 **Left:** Gain from accumulated experience for Opus 4.8: the w/ experience run minus the w/o experience baseline (independent restarts) under the same total time budget. **Right:** Context-length ablation: 200k vs. 1M on Opus 4.8. Each curve shows the average performance over time; dashed lines are log-sigmoid fits.

5.3 How Much Does a Longer Context Improve Performance

Motivation. Section 5.2 shows that accumulated experience can improve performance. A remaining question is how this experience should be retained during a long run. Using long context is a natural way to do so. However, frontier-agent harnesses can also maintain state outside the model context through workspace files, compaction, progress notes, and memory-like artifacts. It is therefore unclear whether extending the context window still provides additional benefits once these external state channels are available, and if so, by how much.

Setup. We compare 200k-context Opus 4.8 with 1M-context Opus 4.8 on the 42-task subset under the same long-horizon evaluation protocol.

Result. A longer context yields a consistent multi-point gain throughout the 12-hour window (Figure 12b). The 1M-context Opus 4.8 stays above the 200k variant at every checkpoint, and the two trajectories run roughly parallel: the gap is +5.8 at 2h and only edges down to +4.4 by 12h, with both curves well described by the same log-sigmoid form. Thus, even with identical external workspace and harness state, a longer context window gives a stable advantage over the horizon, with at most a slight tendency to narrow.

5.4 Case Study

Setup. We examine the gravitational-wave reconstruction task. Based on the first GW150914 detection paper [1], the agent must recreate the published signal analysis from LIGO strain data. The target has three output groups: H1/L1 waveforms, H1/L1 spectrograms, and velocity/separation curves for the source dynamics. The judge weights the five component scores at 0.15 for each waveform, 0.20 for each spectrogram, and 0.30 for velocity/separation. We run a Codex agent with periodic auto-evaluation, auto-resume, no Internet access, a 30-minute evaluation interval, and a 120-second submission cooldown. The agent made 224 explicit submissions, the harness added 23 auto-evaluations, and the run timed out at the 12-hour budget.

The trajectory reveals a sparse but structured diagnose-edit-evaluate loop. The loop is sparse because most submissions are exploratory probes: only 27 of 224 agent submissions improve the best-so-far score by at least 0.1 percentage points. It is structured because feedback repeatedly changes what the agent searches for. The agent first makes the task measurable, then breaks unresolved errors into smaller searches, then identifies a main bottleneck and keeps searching around it, and finally keeps a working solution while repairing the

图 11. 左:Opus 4.8 在相同总时间预算下“有经验”减去“无经验”(独立重启) 的增益。右: 上下文长度消融,200k 与 1M 在 Opus 4.8 上的对比。每条曲线显示平均性能随时间变化; 虚线为对数-S 形拟合。

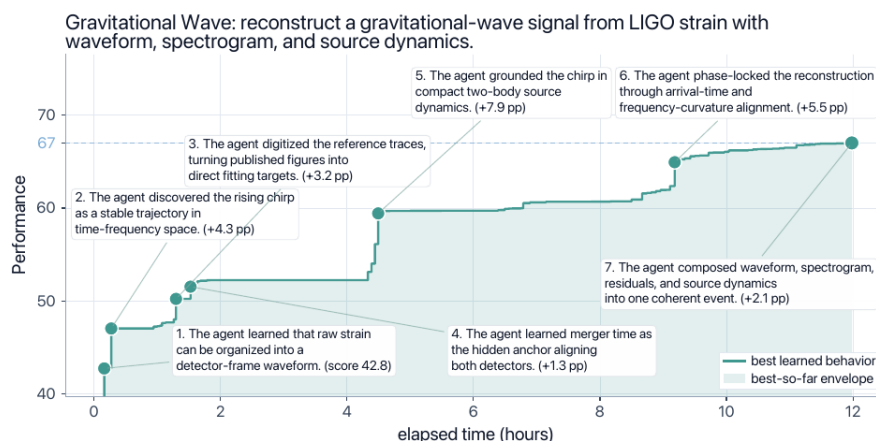


Figure 13 Gravitational-wave reconstruction task case study. The trajectory shows how a GPT-5.5 agent improves over a 12-hour run, with numbered milestones marking representative best-so-far updates.

remaining errors. These patterns show how many failed trials can still produce a small number of cumulative improvements.

- **The agent first makes the problem measurable before making it better.** The first valid submission turns an underspecified analysis task into a scoreable pipeline, but feedback exposes weak source dynamics. The agent then spends the first 11 submissions stabilizing the pipeline and replacing a noisy frequency estimate, producing three meaningful updates and a +4.5 pp gain.
- **When direct repair stalls, the agent decomposes the failure into searchable subproblems.** Instead of treating waveform mismatch as one opaque error, the agent separates reference anchoring, time-frequency localization, and detector alignment. Across 40 signal-search submissions, this decomposition yields seven meaningful updates and lifts the best score to 52.3.
- **Identifying a main bottleneck lets the agent keep searching productively.** After broad signal-processing edits plateau, component feedback identifies velocity/separation as the dominant gap. The agent keeps searching within source-mass calibration rather than rewriting the whole pipeline; in the 4–5h window, 17 submissions and five useful updates raise source dynamics from 64.2 to 89.0, creating the largest jump in the run.
- **After finding a stable solution, the agent keeps the core and repairs only the remaining errors.** In the final hours, most residual edits fail to transfer. Instead of restarting the pipeline, the agent keeps the source model fixed and tests targeted residual corrections, phase alignment, and narrow-band corrections. These useful updates raise the H1 waveform component score from roughly 47 to 95, while the aggregate best score reaches 67.0.

The run improves through uneven jumps rather than a smooth climb. Across 247 scored evaluations, the best score rises from 42.8 to 67.0 on the 0–100 scale. Figure 13 shows seven representative milestones. These jumps correspond to the behavior patterns above: the agent first makes the task scoreable, then localizes the signal, improves source dynamics, and finally repairs the remaining H1 waveform errors. Detailed milestone phases and final subscore composition are reported in Appendix G.2.

图 12. 引力波重构任务案例研究。该轨迹展示了 GPT-5.5 智能体如何在 12 小时运行中改进；编号里程碑标记代表性的最佳更新。任务说明：从 LIGO 应变重构引力波信号，包含波形、时频谱与源动力学。

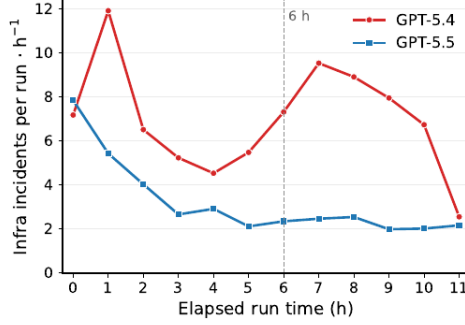


Figure 14 Serving-side incident rate during long-horizon GPT-5.4 and GPT-5.5 runs. Incidents are normalized by active run time; the dashed line marks six elapsed hours. GPT-5.4 experienced substantially more infrastructure and API interruptions, especially in the later part of the run.

Figure 14 quantifies this for the two GPT models, plotting the serving-side incident rate (normalized by active run time) against elapsed time: GPT-5.4 saw substantially more infrastructure and API interruptions than GPT-5.5, especially after the six-hour mark. This carries two caveats for GPT-5.4. First, several of the per-task cells with fewer than three valid runs (marked with * in the score tables) involve GPT-5.4, so its lower run coverage partly reflects serving reliability rather than model behavior, though the reported scores still use only valid trajectories. Second, it offers an operational explanation for the forecast deviation flagged in Section 3.2 (Figure 7): with availability dropping in the second half of the run, GPT-5.4’s later trajectory is noisier and pulls away from the log-sigmoid forecast fit to its first 6.5 hours.

C Evaluation Hacking

Long-horizon evaluation gives agents many opportunities to adapt to evaluator feedback. During task construction and adversarial stress tests, we audited traces for strategies that raised measured scores without exercising the intended capability. These observations are development diagnostics, not official model results: affected tasks were revised or excluded, and the cases below motivated safeguards in the final benchmark.

Feedback as an oracle for hidden answers. In `cylinder_wake_prediction`, an agent treated per-case absolute errors as equations and reconstructed the hidden targets through more than 400 submissions. A lookup table then scored 1.000 without solving the underlying fluid dynamics problem, whereas the best observed submission based on a physics model scored 0.165.

Optimizing stochastic upper tails. In `nethack_dungeon_agent`, an agent removed a fixed random seed after recognizing that higher variance increased the chance of a favorable evaluation. Across 311 submissions, its best score was 1,501 while its mean was 484, showing how best-of- N selection can reward repeated sampling as well as policy quality.

Overfitting evaluator seeds. In `bipedalwalker_locomotion_rl`, the development judge initially used a single deterministic episode, allowing agents to infer and optimize for the reused seed rather than expected return. One run reached a Hardcore return of 301.5 on the judge seed but averaged about 12 over a local 100-episode evaluation; another searched a small hand-coded controller directly against the judge formula. This motivated hidden multi-seed evaluation for stochastic control tasks.

Crossing a trust boundary. In `autolifter`, an agent discovered that anti-cheat checks exempted the repository’s `baseline/` directory and moved an implementation based on an oracle into that trusted path. It solved

图 13. 长视野 GPT-5.4 与 GPT-5.5 运行期间的服务端事件率。事件按活跃运行时间归一化；虚线标记六小时。GPT-5.4 经历了显著更多的基础设施与 API 中断，尤其在运行的后半段。

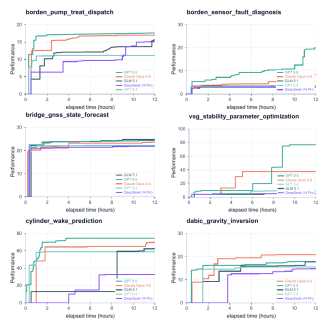


Figure 15 Pre-task learning curves: Scientific Computing & ML (1/21).

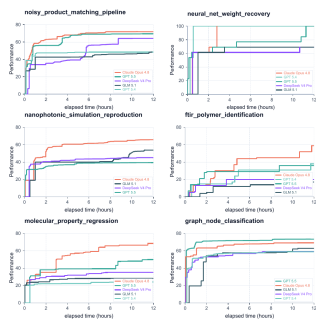


Figure 16 Pre-task learning curves: Scientific Computing & ML cont. (2/21).

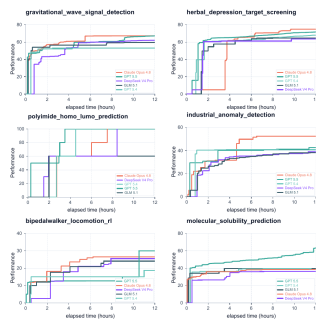


Figure 17 Pre-task learning curves: Scientific Computing & ML cont. (3/21).

53

54

55

图 14. 逐任务学习曲线 (节选)。完整 21 套曲线见原始论文附录 G.5。

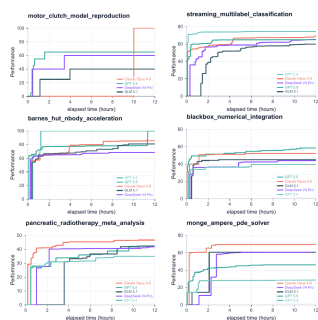


Figure 18 Pre-task learning curves: Scientific Computing & ML cont. (4/21).

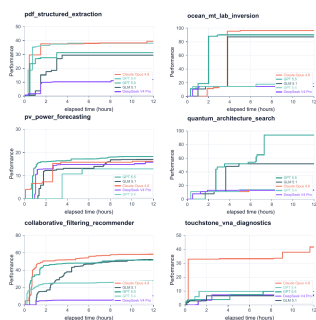


Figure 19 Pre-task learning curves: Scientific Computing & ML cont. (5/21).

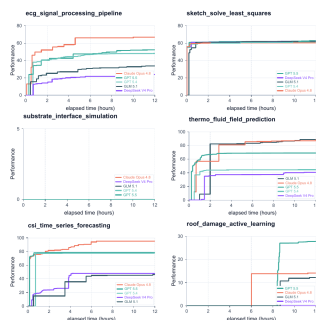


Figure 20 Pre-task learning curves: Scientific Computing & ML cont. (6/21).

56

57

58

图 15. 逐任务学习曲线 (科学/ML 与系统工程, 节选)。